

BLE Security Testing: Survey of Attacks and Evaluation of Tools and Frameworks

HANNAH GREß¹, JONAS SCHREITER², MICHAEL RADEMACHER^{2,3}, AND BJÖRN KRÜGER^{1,4}

¹Department of Epileptology, University of Bonn, University Hospital Bonn, 53127 Bonn, Germany

²Department of Computer Science, Bonn-Rhein-Sieg University of Applied Sciences, 53757 Sankt Augustin, Germany

³Fraunhofer FKIE, 53343 Wachtberg, Germany

⁴Department of Computer Science, University of Bonn, 53115 Bonn, Germany

CORRESPONDING AUTHOR: H. GREß (hannah.gress@ukbonn.de)

This work was supported in part by the Federal Ministry of Research, Technology and Space (BMFTR) of Germany as part of the SecureNeuroAI Project.

ABSTRACT Nowadays, more and more areas of our lives are becoming ‘smart,’ including homes, industrial sites, and medical devices. Since sensitive data is transmitted over various protocols, manufacturers must ensure that the data is adequately secured. To do so, penetration testing on these devices is an option, ideally before market launch. Depending on the protocol, various tools and frameworks exist. We chose the Bluetooth Low Energy (BLE) protocol for analysis due to its widespread use and started our work by classifying possible threats to BLE into four categories based on the attacks executable by the identified tools and frameworks. Threats targeting BLE Mesh and BLE privacy and localization were identified through a literature review and added to the corresponding categories of our taxonomy. Subsequently, we conducted an in-depth evaluation of these tools and frameworks targeting BLE (excluding BLE Mesh, and BLE privacy and localization) on four medical devices to empirically determine which tools and frameworks remain usable for pentesting BLE devices. Furthermore, we analyzed which threats in our taxonomy remain feasible. Our results show that only eight out of 21 (38%) attacks can still be carried out. The tools and frameworks capable of conducting these attacks are mostly still maintained, emphasizing the importance of keeping such tools and frameworks up to date for future use. Nevertheless, current BLE pentesting requires a patchwork of multiple, often unmaintained frameworks to realize all known attacks — this demonstrates an urgent need for a single, extensible, actively maintained framework that allows attacks to be integrated modularly.

INDEX TERMS BLE, Bluetooth low energy, framework, penetration testing, pentesting, survey.

I. INTRODUCTION

THE market for Internet of Things (IoT) devices has grown steadily over recent years and increased from an estimated 7,741 million devices in 2019 to roughly 17.7 billion devices in 2024 (~ 228% growth), with projections suggesting a total increase of about 525% by 2034 [1]. Although the healthcare sector plays a smaller role (estimated at 3.7% in 2019, 4% in 2024, and 5.65% in 2034) the Internet of Medical Things (IoMT) opens up new possibilities [1]. Whether for real-time or remote patient monitoring [2], [3], personalized treatment or preventive care [4], connected devices are transforming the healthcare sector. Although the sensitive data collected by IoMT devices is required to be protected by law, such as Health Insurance Portability and Accountability Act (HIPAA) [5] in the USA or General Data Protection Regulation (GDPR) [6] in Europe, these devices

inherently face many challenges with respect to data storage and transmission. From the device itself, through the app serving as a gateway, and finally to the vendor’s server [7], [8] allowing the monitoring, the data must be adequately protected, since this storage and transmission chain offers a large attack surface.

Cyberattacks are on the rise and do not spare the healthcare sector, as recent data breaches have shown [9], [10]. To protect IoMT devices, penetration testing prior to market entry is recommended, and if vulnerabilities are discovered afterwards, firmware patches for these devices should be provided by the vendor and installed by the user as soon as possible. Given that BLE is widely used by IoMT devices, we selected this protocol to investigate available penetration-testing tools and frameworks in depth.

Recent work has also demonstrated that BLE devices, such as fitness trackers, remain at risk of being hacked by standards-compliant Man-in-the-Middle attacks [11].

We conducted a literature and web search (mostly GitHub) and inspected available Linux packages and utilities to identify tools and frameworks suitable for pentesting BLE devices. We define a tool as a piece of software or a script that can perform one of the attacks we describe as threats in our taxonomy (see Section IV), whereas a framework contains multiple tools and can therefore conduct multiple attacks or an attack chain. In addition, we conducted literature research to identify attacks on BLE Mesh and BLE privacy and localization to further extend our taxonomy to cover recent developments in BLE. We tested each of the tools and frameworks identified as targeting BLE on four different devices, each either medically certified or equipped with sensors allowing monitoring physiological parameters. We evaluated whether the tool or framework could still be set up, and whether all attacks could still be executed successfully. Finally, we summarize our findings and show which attacks on BLE devices are still possible with the tools and frameworks we identified. Therefore, our contributions are:

- Conceptual: A taxonomy classifying threats to BLE, BLE Mesh, and BLE privacy and localization into four phases from advertising to connection.
- Empirical: An in-depth evaluation of identified tools and frameworks targeting BLE (excluding BLE Mesh and BLE privacy and localization) on four devices.
- Based on the empirical evaluation: A summary of findings with an outline of which BLE attacks remain feasible.

The rest of the paper is structured as follows: We start with a review of related work in Section III. In Section IV, we classify the attacks from the analyzed tools and frameworks as well as attacks identified through the literature review on BLE Mesh and BLE privacy and localization into four categories, ranging from advertising to connection. In the subsequent section, we present the tools and frameworks along with the attacks each can execute. Section VI describes the in-depth evaluation of each tool and framework and its results. We summarize our findings in Section VII and conclude in Section VIII.

II. ABBREVIATIONS AND ACRONYMS

AES-CCM	Advanced Encryption Standard - Counter with CBC-MAC.
AoA	Angle of Arrival.
API	Application Programming Interface.
AppKey	Application Key.
ATT	Attribute Profile.
BC	Bluetooth Classic.
BDA	Bluetooth Device Address.
BLE	Bluetooth Low Energy.
CTE	Constant Tone Extension.
CRC	Cyclic Redundancy Check.
CVE	Common Vulnerabilities and Exposures.

CWE	Common Weakness Enumeration.
ECDH	Elliptic Curve Diffie-Hellman.
DevKey	Device Key.
DoS	Denial-of-Service.
DUT	Device Under Test.
FSM	Friendship Security Material.
GAP	Generic Access Profile.
GATT	Generic Attribute Profile.
GDPR	General Data Protection Regulation.
HCI	Host Controller Interface.
HIPAA	Health Insurance Portability and Accountability Act.
HW	Hardware.
IDS	Intrusion Detection System.
IRK	Identity Resolving Key.
IoMT	Internet of Medical Things.
Io(M)T	Internet of (Medical) Things.
IoT	Internet of Things.
JSON	JavaScript Object Notation
LPN	Low Power Node.
MAC	Medium Access Control.
MIC	Message Integrity Check.
MitM	Man-in-the-Middle.
NetKey	Network Key.
NIST	National Institute of Standards and Technology.
NVD	National Vulnerability Database.
OS	Operating System.
OSI	Open Systems Interconnection.
pcap	Packet Capture.
PDU	Protocol Data Unit.
RFID	Radio-Frequency Identification.
PTES	Penetration Testing Execution Standard.
RSSI	Received Signal Strength Indicator.
SDR	Software Defined Radio.
SoC	System on a Chip.
STK	Short Term Key.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial-of-Service, and Elevation of Privileges.
TK	Temporary Key.
TTL	Time to Live.
UUID	Universally Unique Identifier.
VM	Virtual Machine.
WAT	Wearable Activity Tracker.

III. RELATED WORK

In the following, we provide a summary of previous work that surveys threats and attacks in the Internet of (Medical) Things (Io(M)T) and BLE fields. Work related to the *Io(M)T sector* analyzes possible attacks on IoT protocols such as Wi-Fi, Bluetooth Classic (BC), BLE, ZigBee, and Radio-Frequency Identification (RFID) and classifies them in different ways. Common categories are which of the three or five pillars of information security (confidentiality, integrity, authentication, availability, non-repudiation) are compromised [12],

approaches following the three-, five- or seven-layer Open Systems Interconnection (OSI) model [13], [14] or self-defined categories as in [15], [16], and [17]. These works also describe the aims, methods, and impacts of the attacks [13], [14], [15]. In most cases, existing countermeasures or possible solutions are identified to combat the attacks [12], [13], [14], [15], [17]. These solutions include novel technologies like learning-based [13], [14], encryption-based or autonomic approaches [13], as well as blockchain, fog and edge computing propositions [14]. Real-world incidents on IoT systems are presented in [16].

Papers focusing on *security in BLE* and partly combined with BC or BLE Mesh employ more fine-grained schemes for categorization. Classification is done according to the affected layer, either as a three-layer model (physical/firmware/host [18] or physical/data/link layer [19]) or one following the BLE stack layers [20]. Further categorizations include the targeted protocol (BC, BLE, BLE Mesh) [18], the BLE version [20], the compromised information security pillars [19], [20] or the attack domain, e.g., Denial-of-Service (DoS) or surveillance attacks [19]. Căsar et al. [21] classify attacks regarding BLE security. Their four groups consist of (1) attacks resulting from inherent limitations of security and privacy in BLE, (2) exploitable weaknesses of the pairing protocol, (3) attacks against already paired devices and (4) weaknesses due to flaws found in BLE implementations. References [22], [23], and [24] did not apply any categorization. The impact of the attacks [19], [22] and defense and mitigation strategies are also provided [18], [19], [20], [21], [22]. Two papers discuss the application of Intrusion Detection System (IDS) solutions for BLE [20] or BLE and BLE Mesh [24]; two others outline recommendations for users, developers and manufacturers [19], [22]. The applicability of certain BLE attacks is demonstrated by providing case studies and useful tools for BLE security investigation [19] or identifying attacks on Wearable Activity Trackers (WATs) by reviewing existing literature [23].

Table 1 provides an overview of the main evaluation points of the related work compared to ours. The table underlines that none of the related work classified the attacks according to the BLE phases or investigated in depth possible tools and frameworks for BLE security research regarding their feasibility, as done in this work. In addition, our work considers attacks on BLE privacy and localization besides BLE and BLE Mesh which has not been done before. Moreover, our work provides statistics about release dates and peaks, development and maintenance time and attack distribution across all investigated tools and frameworks.

IV. TAXONOMY OF BLUETOOTH LOW ENERGY THREATS

Before providing an in-depth description of different tools and frameworks for BLE, we present a taxonomy of threats applicable to BLE, BLE Mesh and BLE privacy and localization. We define a tool as a piece of software or a script that can perform one of the attacks we describe as threats in our taxonomy, whereas a framework contains multiple tools and

TABLE 1. Comparison of the Related Work With Our Work.

		IoT protocols						Bluetooth							
		[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]	This
Categorization	CIA triad / 5 pillars	X							X	X					
	OSI layer model		X	X											
	Affected layer							X	X	X					
	Targeted protocol							X							X
	Bluetooth version									X					
	Attack domain								X						
	BLE security related											X			
	BLE phases														X
	Self-defined			X	X	X					X				
	No classification												X	X	X
Aims / Methods / Impact		X	X	X					X			X			
Counter-m.	Existing countermeasures	X	X	X	X		X	X	X	X	X	X			
	Novel countermeasures		X	X											
	IDSs									X					X
	Recommendations								X			X			
Applic.	Case studies				X				X				X		
	Tool presentation								X				X		
	Tool testing & evaluation														X
	Tool statistics														X

can therefore conduct multiple attacks or an attack chain. We introduce a subsection on threats to BLE privacy and localization, because attacks in this subsection do not target BLE communication or cryptographic mechanisms, unlike threats to BLE and BLE Mesh, but instead exploit physical-layer sensing or identifier management to compromise device privacy. The differentiation between BLE and BLE Mesh stems from their different use cases, protocol stacks, and specifications.

We outline our classification in Fig. 1. Threats targeting BLE base on tools and frameworks we found through literature and web research as well as inspection of available Linux utilities and packages. We conducted a literature review on *Google Scholar* with the search term:

```
(IoT OR BLE OR Bluetooth Low
Energy) AND
(vulnerability OR security) AND
(pentesting OR testing OR
assessment OR
framework)
```

GitHub results were obtained by searching for tools and frameworks related to the threats identified in the literature research. For the web search, we considered *GitHub* projects which are rated with more than 100 stars. Linux packages and utilities were identified by reviewing the available packages on Kali Linux dealing with Bluetooth or BLE. As described in Section I, we further filtered out Bluetooth Classic-only packages and utilities. Attacks targeting BLE Mesh and BLE privacy and localization were identified through a literature review on *Google Scholar* with different search terms and filter criteria: BLE Mesh AND (attack OR vulnerability), considering articles published between 2020 and 2025; BLE Direction Finding AND (attack OR vulnerability), considering articles published between 2017 and 2025; BLE Channel

Sounding AND (attack OR vulnerability) considering articles published between 2024 and 2025; and BLE MAC Randomization AND (attack OR vulnerability), considering the first 100 articles.

Following the *Bluetooth Low Energy operations* defined in the Bluetooth specification, we group the threats into four main categories, which correspond to different stages in the connection of a BLE device [25, 1.A.1.2]: (1) **advertisement**, (2) **connection establishment between two legitimate devices**, (3) **connection between an attacker and a legitimate device** and (4) **connection between two legitimate devices**. In our classification, establishing or being in a connection does not necessarily mean that the two devices are paired. But some of the attacks in these categories are only possible when the two devices are already paired, either with the attacker or with the other legitimate device.

A. BLUETOOTH LOW ENERGY

The first category, **advertising**, corresponds to the initial phase of connection establishment. During this phase, a BLE device – later becoming the Peripheral – broadcasts advertisement packets to signal its availability for connections. Meanwhile, a BLE device that will later serve as the Central, scans for these advertisement packets to initiate a connection. An attacker can leverage this phase by scanning for advertisement packets, too, to gather information about nearby devices.

From our research, we identified eight different attacks an attacker can carry out during the advertisement phase. First, the attacker can gather as much information as feasible about a BLE device to be able to conduct a vulnerability scan in publicly available databases (e.g. Common Vulnerabilities and Exposures (CVE), Common Weakness Enumeration (CWE), National Vulnerability Database (NVD)) to identify known weaknesses in the specific device. Sources of information might be the Bluetooth Device Addresses (BDAs) of the targeted device, information extracted from enumerating its services and characteristics (e.g. its device name, manufacturer name or device model number) and the Bluetooth product listing. The Bluetooth product listing contains all devices and products that have completed the Bluetooth Qualification Process and are therefore allowed to use any of the Bluetooth trademarks. The listing for each product contains its manufacturer name, device serial number, hardware, firmware or software revision string and the product's referenced qualified design that is a prerequisite for the first attack, conducting a vulnerability search [26], [27]. Next, the attacker can collect the BDAs or other static device identifiers, for example static byte sequences in the advertisement data, to perform two different attacks, device fingerprinting, i.e. the identification of unique information that characterizes a specific device, or long-term tracking [28], [29], [30], [31]. The fourth possible attack is BDAs spoofing [32], [33], [34], [35], [36], [37] and to extend this attack, the spoofing of advertisement data can additionally be done as a fifth attack. For that, the attacker clones the

advertisement data of the device and its name and advertises itself using the fake data [26], [32], [33], [34], [36], [37], [38], [39], [40]. The attacker can advertise themselves faster than the spoofed device to increase the chance that they become connected to the other legitimate device instead of the spoofed one [26], [34]. The sixth identified attack can only be applied to Apple devices. In this case, the attacker can dissect the advertisement packets of an Apple device to gain additional information such as model name, color and battery status coded into the advertisement data [29], [30]. For iOS versions after 2021, the dissection is only partially possible [41]. Seventh, the attacker can measure the device's uptime by storing the time stamp when the attacker has first seen the device, meaning when the attacker received the first advertisement packet [29]. This might not be the actual uptime, especially when the attacker or the target device is not steady but moving around. Lastly, the attacker can use the Received Signal Strength Indicator (RSSI) value as a prerequisite to estimate the proximity between them and the victim device [30].

The second category, **connection establishment between two legitimate devices**, includes the exchange of the connection handshake. We found one possible attack an attacker can conduct during this stage. As a prerequisite, the attacker sniffs the pairing handshake [33], [36], [37], [42], [43] and — if the devices used Legacy Pairing — brute-force the Temporary Key (TK) under a second. Brute-forcing the TK enables the attacker to decrypt and thus read the packets sent between the legitimate devices, to modify them or to even inject malicious packets [33], [34], [36], [37], [44].

The third category is the **connection between the attacker and a legitimate device**. We discovered seven different attack possibilities for this category. An attacker can perform Service Discovery, discovering the services and characteristics of a legitimate device, together with their possible operations (read, write, notify, indicate) [26], [34], [35], [37], [36], [39], [40], [45], [46], [47], [48], [49]. This information can be used for a first attack, executing read, write or subscribe operations [36], [45], [46], [47], [48], [50], fuzzing a device, to observe its behavior when non-conforming data is written to its characteristics [26], [33], [34], [48], [51] or conforming data at the wrong time [51]. The Bluetooth specification defines a set of services and characteristics [52] and when used by a device, the attacker can exploit this publicly available knowledge to attack in a more targeted way. If a service is vendor specific, sniffing and analyzing the traffic might reveal for which operation(s) the device uses the service [53, p.297-305]. The second attack possibility is the discovery of the minimum level of security for writing and reading a characteristic [54]. Besides the access permission, a characteristic can have encryption, authentication, or authorization permission. The so-called attribute permissions are defined by a higher layer specification and not the ATT layer, but can be discovered when connected to a device [25, 3.F.3.2.5]. As a third attack, an attacker can impersonate a device by creating a fake GATT

Server, thereby cloning its services, characteristics and maybe the characteristic values [33], [34], [36], [39], [40], [45], [49], [55]. Together with the spoofing of advertisement data, an attacker can disguise themselves almost perfectly as the spoofed device. Either one or both of the legitimate devices can be spoofed. When both are spoofed and the attacker holds a connection to both of them, they can read, modify or inject packets into the connection [26], [37], [56]. Two further attacks possible when the attacker is connected to both spoofed devices are packet relay and packet tampering [26], [33], [34], [37]. Another attack would be a replay attack [26], [34], [37], [36], [57]. Here, the attacker needs only to be connected to one of the devices. As an additional attack, an attacker can perform a DoS attack. Such an attack may take different forms; for example, an attacker can keep a victim device busy by sending read, write, or connection requests, making it difficult or impossible for its legitimate pairing partner to communicate with it. Furthermore, an attacker can transform this attack into a battery-draining attack when sending the requests over a long period of time resulting in unavailability of the victim device due to an empty battery [27], [34], [33], [42]. On top of that, a BLE device acting as Peripheral allows only one active connection with another device; consequently, if it is connected to the attacker device, it becomes non-visible and non-connectable towards the legitimate pairing partner [25], [58], [59, 1.A.6.2]. Moreover, an attacker can jam the frequency band, thereby disturbing the physical layer making it hard or impossible to communicate with the jammed device [33], [34].

For our last category, the **connection between two legitimate devices**, we identified five different attack possibilities. First, the attacker can sniff the ongoing connection between the legitimate devices [27], [33], [34], [42], [43], [36], [60], [61]. If the devices do not use encryption, or if they only use Legacy Pairing and the attacker has recorded the pairing handshake, the attacker can also read the exchanged packets [36], [42]. An attacker can inject falsified packets under the same conditions [34], [56]. As a third attack, the attacker can jam the connection, which might serve as a preparation for further attacks [61]. Fourth, the attacker can hijack an ongoing BLE 4.x connection. Given the access address and the channel map of the device, the attacker can synchronize with the connection after calculating the Cyclic Redundancy Check (CRC) Init value, the hop interval and the hop increment, and can then take it over [44], [61]. Finally, an attacker can analyze the traffic with respect to message frequency [29].

B. BLUETOOTH LOW ENERGY MESH

Since BLE mesh is built on top of BLE, BLE Mesh reuses core functionalities and processes of BLE [62, 2.1]. Therefore, we first analyze the applicability of the identified attacks on BLE Mesh and then describe attacks only possible on BLE Mesh and group them into the respective category of the taxonomy. The **advertising** phase in BLE Mesh is the same as in BLE except that a so-called *unprovisioned* BLE device,

i.e., a device intending to join the mesh network, includes the *Mesh Beacon* AD type in the advertising packet [63, 4.16.1]. Therefore, all prerequisites and attacks in the advertising phase outlined in the taxonomy are theoretically applicable to BLE Mesh. However, little work exists empirically validating this assumption. Prior work mentions BDAs and advertisement spoofing in BLE Mesh [64] and performing proximity estimation via RSSI strength measurement [65]. But it should be noted that the proximity estimation is done with BLE devices already part of the mesh network, so provisioned nodes. Since BLE Mesh communication primarily relies on advertising messages (advertisement bearer), the assumption of RSSI-based proximity estimation still holds when the node measuring the RSSI value is a direct neighbor of the sending node [62, 2.1.7, 3.3.1]. A limitation of this setup is that an attacker can hardly gain access to a BLE Mesh network due to its mandatory and strong security mechanisms [18], [66] and that nodes relaying messages do not include their BDAs, making it impossible to clearly identify each node. However, research has shown that it is possible for an attacker to access a network illegitimately under certain conditions [67], [68].

Since BLE mesh uses the Elliptic Curve Diffie-Hellman (ECDH) protocol for key establishment, sniffing the pairing handshake and brute-forcing of the TK are not possible during the **connection establishment between legitimate devices** [62, 5]. However, other attacks on BLE mesh are possible during the provisioning process, i.e., adding a new BLE device to the mesh network. Due to the similarity between the connection establishment and provisioning, we group these attacks into a single category. The first attack on the provisioning protocol is a reflection attack. To this end, the attacker first performs a Man-in-the-Middle (MitM) attack during the ECDH key exchange resulting in a shared ECDH secret with both the provisioner and the BLE device willing to join the network. Then, the attacker reflects all values exchanged during the authentication procedure back to the provisioner and completes the provisioning process. The attacker is now part of the network, but the legitimate device is not provisioned [64], [68]. The second attack exploits a default *AuthValue* with low entropy. The *AuthValue* is the Authentication Data padded with zeros and a parameter of the authentication commitment protocol. In this attack, the attacker performs a MitM attack during the ECDH key exchange, starts the authentication procedure with the provisioner, interrupts after receiving the relevant values (confirmation value and nonce) from the provisioner and then brute forces the *AuthValue*. When the user restarts the provisioning process, the attacker again becomes the MitM and completes the provisioning process with each legitimate device separately, since the attacker is in possession of all relevant parameters obtained from the previous provisioning process [68]. The third attack is possible due to a malleable commitment protocol. The attack starts similarly to the previous one, but instead of interrupting the provision process, the attacker recovers the *AuthValue* as described in *M-R4* and *M-R5* in [68]. The

provisioning process can be completed successfully and the BLE device is provisioned to the network. In the latter two cases, the attacker must remain online throughout to relay the messages sent between the BLE device and the mesh network. In all cases, the attacker possesses the Network Key (NetKey), potentially the Application Key (AppKey) and the device specific Device Key (DevKey) of the provisioned BLE device [68].

BLE devices that do not include the BLE mesh stack can still participate in a mesh network via Generic Attribute Profile (GATT). In this case, a proxy node and the BLE device both implement the required GATT services (Mesh Provisioning Service and the Mesh Proxy Service). The proxy node transforms the GATT Protocol Data Units (PDUs) into mesh network PDUs and vice versa, enabling communication. Every BLE device using GATT for BLE mesh communication must first be provisioned, as is the case for all nodes [63, 4.18.3.1], [62, 7]. Possible GATT-related attacks during a **connection between an attacker and a victim device** in a BLE mesh network are subject to future research. Since packet relay is the basis of communication in BLE mesh networks and enables extending the range of communication [62, 2.2], an attacker can exploit it to conduct selective forwarding attacks. In this case, the malicious node either forwards selected messages and drops others (gray hole attack) or does not relay any of them (black hole attack). Due to the managed flooding approach and the many-to-many communication in BLE mesh, the effectiveness of the attack depends on the number of malicious nodes, the network density and the role of each node, especially if the network contains a significant number of Low Power Nodes (LPNss) and Friend nodes [20]. Another attack, packet tampering, is partly prevented by message encryption and authentication using Advanced Encryption Standard - Counter with CBC-MAC (AES-CCM) on the upper transport and the network layer. On the upper transport layer, the application nonce and the AppKey or the device nonce and the DevKey are used. On the network layer, the network nonce and the EncryptionKey derived from the NetKey is used. In all three cases, the nonce contains, among others, the sequence number, the source address, and the IV Index [62, 3.9.5.1 - 3.9.5.3, 3.9.7.1-3.9.7.2]. Since AES-CCM needs the same parameters for encryption and decryption, an attacker possessing the same NetKey and AppKey can encrypt, modify, re-encrypt, and forward a packet, unless the sender transmits another packet with a higher sequence number in the meantime, leading to the discard of the modified packet [63, 7.6.1]. If the message is encrypted on the upper transport layer using the DevKey and the Configuration Manager is not malicious, the message is protected against packet tampering, since the DevKey is known only to the device and the Configuration Manager [62, 3.9.6]. In all other cases, data manipulation results in an invalid Message Integrity Check (MIC) and the message shall be discarded [63, 6.3]. Packet replay is prevented, as each message contains a nonce that includes the so-called IVISeq value. This is composed of the IV Index and the

sequence number of the message which is incremented with each message. Message integrity checks on each message, either on the upper transport or network layer (TransMIC and NetMIC, respectively), reveal replay attacks [62, 3.9.8]. BLE mesh networks may be susceptible to DoS attacks. The managed flooding approach for sending packets limits the likelihood of DoS attacks by introducing a Time to Live (TTL) for each packet and requiring nodes to cache recently forwarded packets to prevent retransmission. In addition, the directed forwarding approach ensures that only a subset of nodes relay a message on a predefined path from sender to receiver reducing bandwidth usage within the network [62, 2.2, 2.3.11]. However, Jürgens et. al [69] demonstrated that network performance can decrease significantly when using the managed flooding approach even without exceeding the limit set by the specification of 100 PDUs transmitted per node per ten seconds. This finding indicates that a high relay frequency of packets can partially or fully degrade mesh network functionality. Moreover, malformed heartbeat messages can also cause network disruption or DoS attacks. Nodes in a mesh network periodically send so-called heartbeat messages to announce their presence and to enable other nodes to estimate the number of hops to that node. An attacker can send malformed heartbeat messages with a reduced TTL claiming to originate from a specific node in the network. Consequently, packets addressed to this victim node may not arrive, as they are discarded during transmission when the TTL reaches 0. Therefore, the network may become congested, potentially leading to a DoS attack [20]. Another attack on BLE mesh networks is a Denial-of-Friendship attack. The concept of friendship and friend nodes applies to LPNss, i.e., power-constrained devices, to reduce their listening time by going into power-saving mode for most of the time. Since the LPNss miss many packets sent in the network, they establish a friendship with another node that caches all packets relevant to it. The friend node transmits the packets as soon as the LPNs wakes up and polls new messages. A friendship can be terminated using the control message *FriendClear*, issued either by the LPNs itself or the new friend node. An attacker can exploit this fact by sending a *FriendClear* message to the current friend node [70]. A *FriendClear* message contains the unicast address of the LPNs being removed (*LPNAddress*) and the *LPNCounter* which is the value of the latest *Friend Request* the LPNs used to establish its current connection [62, 3.6.5.5]. In addition, the attacker needs the address of the current Friend node so that the *FriendClear* message reaches the correct node. To this end, the attacker sends a *Friend Request* message into the network to receive the Friend address and the *FriendCounter* from all Friend nodes in proximity. Finally, the attacker must determine the LPNs's address and its *LPNCounter*. Since the *FriendCounter* is typically higher than the *LPNCounter*, and unlike LPNss, Friend nodes can keep multiple friendships, the attacker can narrow down the range to search for the *LPNCounter*. The search is performed by brute force. As a result of a Denial-of-Friendship attack, the LPNs must enter

the scanning state again to search for a new Friend node, increasing its battery consumption [70].

When the attacker is located outside the BLE mesh network and there is a **connection between legitimate devices**, sniffing is not possible due to mandatory double encryption at the upper transport and network layers [62, 3.9.7.1-3.9.7.2], [66]. The same applies to packet injection, since the attacker does not possess the keys for encryption and authentication. Connection jamming is possible and was successfully shown by Kalinin et al. [71]. Session hijacking is not possible, since communication between BLE and BLE Mesh devices differs; therefore, session hijacking procedures are not applicable. Message frequency measurement is possible with limitations. Due to the connectionless nature of a mesh network, message frequency between individual nodes cannot be measured. Network layer obfuscation makes tracking or identification of individual nodes difficult, but not impossible. With sufficient effort, an attacker may discover patterns in this obfuscation that could reveal the source address or sequence number of a node. Thus, an attacker could at least count packets originating from such a node. Additionally, the potentially large size of a network makes it difficult to observe every packet or even identify every node participating in the network, complicating comprehensive observation of transmitted messages in the network [62, 2.1.7, 2.2, 3.9.7.3]. Two further attacks on BLE Mesh result from the impersonation of a LPNs by obtaining the Friendship Security Material (FSM) [70]. The FSM is determined by five parameters: NetKey, LPNs address, Friend address, *LPNCounter* and *FriendCounter*. The attacker possesses at least the NetKey, allowing the attacker to sniff all communication in the network, including friendship establishment procedures. By receiving a Friend Request message, the attacker has the LPNs address and the *LPNCounter*. By filtering all Friend Offer messages addressed to the LPNs, the attacker obtains the Friend node's address and *FriendCounter* and thus all parameters required to compute the FSM. The first resulting attack is the *malicious queue friend*. Here, the attacker sends a Friend Subscription List Add message to the Friend node to add a subscription to a new group address while the legitimate LPNs is in power-safe mode. As a result, the Friend node caches all packets sent from this group address for subsequent relay to the LPNs. Depending on how many group addresses the Friend node has subscribed to and how many messages are sent in total to the Friend node for caching, the cache of the Friend node might become saturated, risking deletion of valid messages for the LPNs. Rather than terminating the friendship, the attacker aims to isolate the LPNs with this attack [70]. The second resulting attack exploits a *high(er) sequence number*. To this end, the attacker sends a message secured with the FSM to the Friend node with a higher sequence number than the last message sent by the LPNs. When the LPNs wakes up and polls the stored messages, the Friend node will ignore and discard this message, because the sequence number is lower than that of the previous message sent from the attacker. As the victim node receives no reply, the *PollTimeout* expires, and

the friendship terminates. This attack is therefore termed a Denial-of-Friendship attack [70]. Even without knowledge of the FSM, an attacker can conduct a Denial-of-Friendship attack. The attacker exploits the fact that the packet size of the *Friend Poll* control messages is small and of unique length. Only the Friend Subscription List Confirm control messages are of equal size, but rarely sent. Thus, an attacker can eavesdrop on the communication to identify the *Friend Poll* messages by inspecting the packet header containing the length field (*AD Length*). If the *AD length* reveals that the size of the message is 19 bytes, the attacker can selectively jam this and subsequent packets thereby terminating the connection between the Friend node and the LPNs. As a result, the LPNs must re-enter the scanning state to search for a new Friend node, increasing its battery consumption [70].

C. BLUETOOTH LOW ENERGY PRIVACY AND LOCALIZATION

Attacks in this subsection do not target BLE communication or cryptographic mechanisms, but instead exploit physical-layer sensing or identifier management to compromise device privacy.

1) PRIVACY RISKS IN MEDIUM ACCESS CONTROL (MAC) RANDOMIZATION

Although MAC randomization aims to prevent device tracking, research has shown that user privacy can still be compromised during **advertising**. A simple tracking approach exploits the static data in the advertising data to uniquely identify a device despite regular BDAs changes [31]. The same applies to the content of *SCAN_RSP* packets. Typically, an Observer sends a *SCAN_REQ* in response to an advertisement packet requesting more information from the advertising device. An attack can exploit the *SCAN_REQ-SCAN_RSP* procedure for unique device identification [72]. Pairing with a device using MAC randomization compromises user privacy, since the Identity Resolving Key (IRK) for resolving the BDAs is device-specific and once obtained, the BDAs can be resolved indefinitely and thus enables device identification [25, 3.H.2.4.2.1]. Another approach is an allowlist-based side-channel attack where the attacker can identify BLE devices using an allowlist based on their behavior. Three allowlist types can be distinguished. A Central using the *scanning filter policy*, or a Peripheral using the *advertising filter policy* only responds to *SCAN_REQ* from allowlisted devices. A Central using the *initiator filter policy* connects automatically to already paired Peripherals when they are in proximity. Depending on the policy, attacks can be passive or active. In a passive attack, the attacker observes whether *SCAN_REQs* from a Central are answered by a Peripheral. If the Peripheral replies only to a specific Central, it uses the *advertising filter policy* and both devices can be identified despite using a randomized BDAs. The *SCAN_REQ* contains both addresses, whereas the *SCAN_RSP* contains only the Peripheral BDAs; correlating *SCAN_REQs* with the resulting *SCAN_RSPs* reveals both addresses. Even if both

devices change their BDAs regularly, address changes are not synchronized and therefore, the outlined tracking mechanism still works. The active attack can be conducted by a malicious Central or Peripheral. As a malicious Central, the attacker acts as in the passive case and records the relevant packets. At a later time, the attacker replays the recorded `SCAN_REQs` to nearby Peripherals. If an allowlisting Peripheral is in range, it will reply to the `SCAN_REQ`, as it resolves and recognizes the Central's BDAs. Because BDAs lack replay protection and only a single Central can appear in a Peripheral's allowlist, the Peripheral can be uniquely identified. As a malicious Peripheral, the attacker targets Centrals using the *initiator filter policy*. To identify such Centrals, the attacker transmits spoofed advertisement packets impersonating a legitimate Peripheral. Upon reception of these packets, the Central initiates a connection and the attacker can subsequently track it.

2) DIRECTION FINDING IN BLUETOOTH LOW ENERGY

For Direction Finding, we identified two attacks aiming to manipulate the measured Angle of Arrival (AoA) by tampering with packets sent by the transmitter. Direction Finding can be used during both advertisement and connections between two devices [25, 1.A.3.3.2]; therefore, we categorize the attacks in the two phases **advertisement** and **connection between two legitimate devices**. In the first attack, the attacker modifies the phase of the packet Constant Tone Extension (CTE). In a normal setup, the transmitter sends out packets ending with a CTE indicating support for AoA [73]. The CTE starts with a guard period (4 μ s), followed by a reference period (8 μ s) and an alternating sequence of switch and sample slots (4–148 μ s) of 1 or 2 μ s [25, 2.5.1]. A receiver uses the switch slots to switch from one antenna to the other and the sample slots to acquire IQ samples to determine the phase of the signal. Consider a setup with two antennas. When a signal is received by the antenna a_1 at time t_1 , the receiver switches to antenna a_2 during the next switch slot, measures the phase at antenna a_2 and estimates the phase at antenna a_1 during the next sample slot at time t_2 . The angle is computed by subtracting the phase of antenna a_2 from the phase of antenna a_1 . An attacker can exploit the fact that the phase for antenna a_1 at time t_2 is only estimated by modifying the phase at antenna a_2 at time t_2 , resulting in an incorrectly estimated AoA for the transmitter [73]. In the second attack, the attacker modifies the CTE reference period signal resulting in a different AoA. This attack applies only to Direction Finding during advertising. First, the attacker monitors the BLE traffic to identify the extended advertising packets `ADV_EXT_IND` (*Extended Advertising Indication*). If the `AuxPtr` field is present in the extended header field of the advertisement packet, the attacker can extract from this field the channel on which the auxiliary packet will be transmitted. If the extended header contains the `SyncInfo` field, it signals the presence of a periodic advertising train using either `AUX_SYNC_IND` or `AUX_SYNC_SUBEVENT_IND` PDUs

[25, 6.B.2.3.1, 2.3.4.5–2.3.4.6]. Therefore, the attacker only needs to switch to this channel and listen for `AUX_SYNC_IND` packets; once one is captured, synchronization with the periodic advertisement train is achieved. The `AUX_SYNC_IND` packet includes a CTE. The attacker can then derive the transmission interval and other parameters of the `AUX_SYNC_IND` packet stream, craft illegitimate `AUX_SYNC_IND` packets with modified reference period signals and inject them at the correct time to falsify the victim device's AoA [74]. Both attacks are possible because the CTE is appended to the packet after optional encryption and is not included in the CRC or MIC calculations [25, 6.F.4.1.7].

At the time of writing, there were no published attacks on Channel Sounding.

V. TOOLS AND FRAMEWORKS

In this section, we provide a comprehensive overview of the tools, frameworks, and Linux packages and utilities from the `bluez` or `bluez-utils` package we identified to create the taxonomy depicted in Figure 1 and described in Section IV. Due to the lack of public available code, we focus on attacks only applicable to BLE in the remainder of the paper. We start the section by introducing low-cost Hardware (HW) sniffers that are used by several of the tools and frameworks presented later. Then, we describe the tools and frameworks that are either closed source or based on prior research for which no source code has been released, followed by those whose code is open-source. We further divide the last open-source subsection into three parts. First, we describe Linux packages and utilities that can be used for different BLE attacks; second, we present tools and frameworks that run only on Linux; and third, we cover those that run on other hardware or Operating Systems (OSes). We conclude this section by introducing some mobile solutions.

A. HARDWARE TOOLS FOR PENETRATION TESTING AND VULNERABILITY ASSESSMENT

In addition to the tools and frameworks presented later, several affordable and widely used hardware tools support BLE analysis and testing.

The *Ubertooth One* was the first low-cost Bluetooth sniffer launched in 2010 when only Bluetooth Classic existed. The open-source device can capture Bluetooth Classic and BLE traffic, optionally with Wireshark, and inject advertisement packets from a specific BDAs. The project was retired at the end of 2022 [75], [76].

Adafruit offers the Bluefruit LE Sniffer, which supports only BLE version 4.0. It uses the nRF51822 from Nordic Semiconductor which supports BLE up to version 4.2 [77], [78].

Nordic Semiconductor offers a wide range of dongles and development boards supporting the newest versions of Bluetooth. At the time of writing, the nRF52, nRF53 and nRF54 Series are the most recent [79].

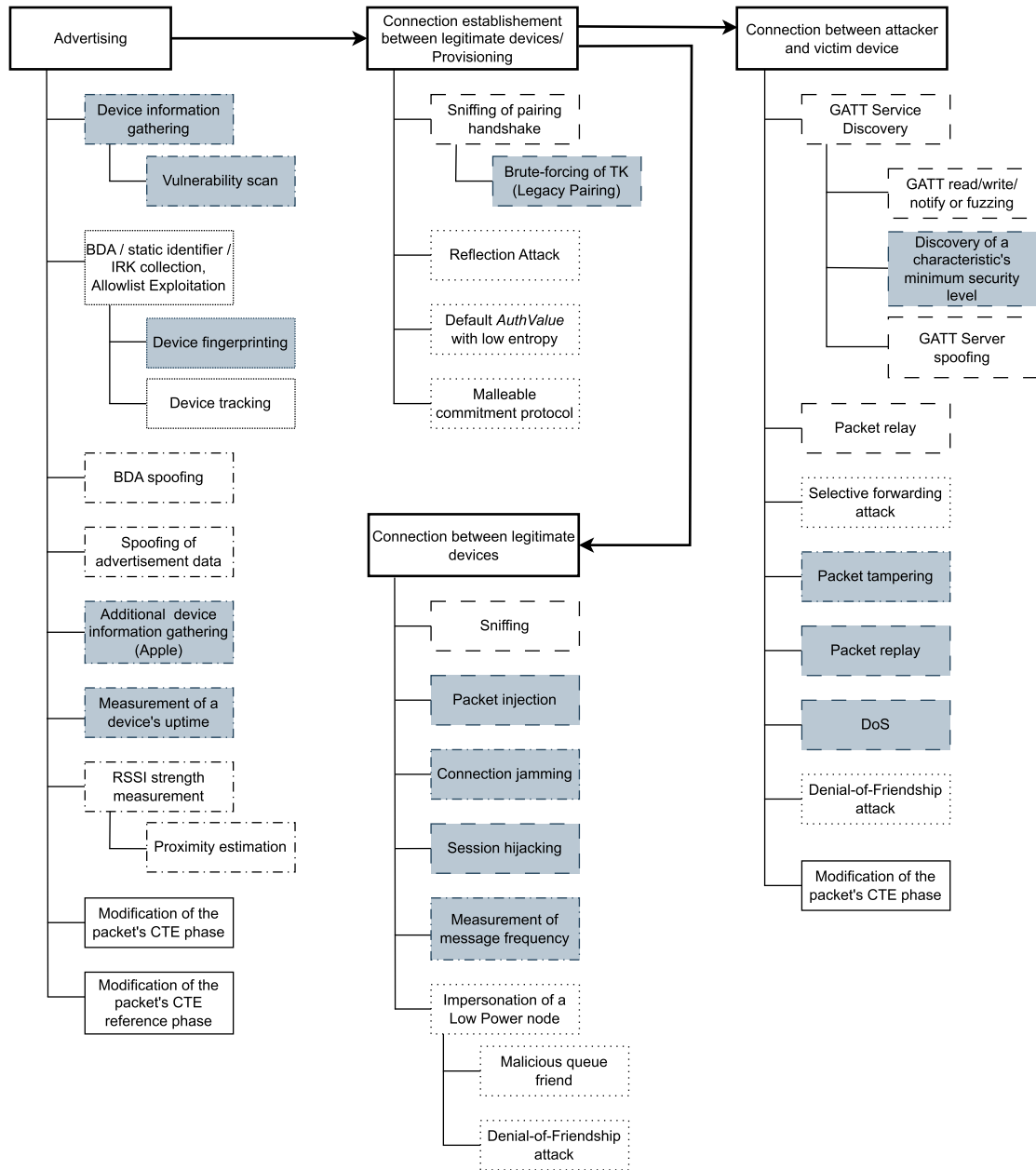


FIGURE 1. Taxonomy of BLE phases and their possible threats. Attacks specific to BLE are indicated by dashed lines. Attacks specific to BLE Mesh (theoretical or empirically validated) are shown with coarse dotted lines. Attacks applicable to both are marked with dash-dot lines. Attacks related to BLE privacy and localization are shown with solid lines, while attacks applicable to all categories are indicated by fine dotted lines. Attacks with a gray background are no longer feasible based on our evaluation of tools and frameworks for BLE-only attacks. Attacks on BLE mesh and BLE privacy and localization were not included in the evaluation.

B. CLOSED-SOURCE TOOLS AND FRAMEWORKS OR WITHOUT PUBLICLY AVAILABLE SOURCE CODE

In this subsection, we discuss one framework for IoT devices supporting multiple protocols, two BLE penetration testing frameworks, one BLE fuzzing tool, and one BLE sniffing tool.

The IoT framework of Siboni et al. [27] combines existing penetration tools with self-developed machine learning-based algorithms for monitoring and analysis and real-world simulators and simulation tools. It offers a vulnerability scan by

querying the NVD. Although it primarily targets Wi-Fi, it includes for BLE a sniffing tool using the *Ubertooth One* and a DoS plugin. This penetration testing frameworks is based on well-known guidelines or models, which simplified its development.

For their so-called *Bluetooth Low Energy Devices Security Testing Framework*, Ray et al. [34] first evaluated the National Institute of Standards and Technology (NIST) Guide to Bluetooth Security [80] for threats concerning Bluetooth versions 4.0 to 4.2 and devices using *Just Works* as

association model. The identified threats resulted in a small testing framework that enables BDAs cloning, fuzzing, GATT Server spoofing, relay and DoS attacks.

Christopher Skallak and Silvie Schmidt [33] provide an overview of different BLE attacks categorized according to the Spoofing, Tampering, Repudiation, Information Disclosure, Denial-of-Service, and Elevation of Privileges (STRIDE) model. After briefly describing each attack, they outline their plan for an open-source, user-friendly BLE development and penetration testing tool called *BLEBerry*.

The only identified fuzzing framework [51] allows two operations. It can either send malformed packets or valid packets at the wrong time or controller state, both of which may lead to unresponsiveness or a non-compliant response from the target device, indicating a potential vulnerability in its Bluetooth implementation.

Nordic Semiconductor offers the *nRF Sniffer for Bluetooth LE*. It captures BLE traffic and exports it to *Wireshark* for analysis [81], [82]. It works with the nRF development kits or dongles.

C. TOOLS AND FRAMEWORKS WITH PUBLICLY AVAILABLE SOURCE CODE

1) PACKAGES AND UTILITIES BASED ON THE LINUX BLUETOOTH STACK BLUEZ

BlueZ is the official Bluetooth stack for many Linux distributions and compatible with all major Linux distributions [83]. The *bluez* or the *bluez-utils* package (depends on the distribution) contains different utilities for testing, debugging, or developing Bluetooth applications. These utilities can also be used for security testing, as outlined below. Moreover, Linux provides packages that directly support Bluetooth attacks, as we show later. Both the packages and utilities can be used for Bluetooth Classic and, in many cases, also for BLE.

We identified 18 packages and utilities in total, of which six were excluded (*blueranger*, *bluesnarfer*, *btscanner*, *l2ping*, *redfang*, *spooftooph*), because they work only with Bluetooth Classic devices. We also excluded *apple-bleee*, since it targets Apple devices, whereas our test devices (described in Section A) are non-Apple devices. Five packages or utilities have been deprecated since 2017 (*gatttool*, *hciconfig*, *hcidump*, *hcidtool*, *sdptool*) and were therefore not taken into account [84]. The utility *btmon* was also excluded, as it did not fit our use case. It is a useful tool when recording the Bluetooth traffic between a remote device and a computer, but for us, the traffic between our test devices and a mobile phone was of importance. We also did not test the package *btmgmt*, because its functionalities can also be found in the submenu *mgmt* of the utility *bluetoothctl*. We therefore describe only four packages or utilities that we consider useful for assessing the security of BLE devices. In some cases, multiple commands from different utilities must be combined to launch an attack, as shown in Section I.

a: BLUETOOTHCTL

can be used to configure the Bluetooth adapter or chip. This comprises, for example, setting the I/O capability, the device name or physical transport mode (discoverable or pairable). It also provides submenus for advertisement, scanning, GATT layer operations, and more.

b: BLUE-HYDRA

is a package for monitoring and tracking nearby Bluetooth devices (both Classic and BLE) [85].

c: UBERTOOTH

offers the necessary library to work with the *Ubertooth One* hardware sniffer described in Section A [86].

d: CRACKLE

exploits a flaw in the Legacy Pairing process of two BLE devices where the only secret of the connection is a 6-digit value that serves as the basis for encrypted communication. Given a Packet Capture (pcap) file containing the complete pairing handshake, crackle can brute-force the secret in under a second [44], [87].

2) TOOLS AND FRAMEWORKS ONLY RUNNING ON LINUX

In this subsection, we introduce tools and frameworks that run only on Linux or its derivatives. We can divide them into tools and frameworks for wireless IoT protocols and BLE-specific ones.

Five of the identified frameworks offer pentesting tools for different wireless communication protocols used by IoT devices. Two of them, *EXPLIoT* and *HomePwn*, support scanning for BLE devices, performing Service Discovery, and reading values from or writing values to characteristics. *HomePwn* also provides tools for device spoofing and traffic sniffing. The sniffed traffic can also be saved into a pcap file [35], [48]. With the framework *SecuWear* [42] sniffing of BLE traffic, injecting (falsified) packets and conducting two different types of DoS attacks is possible. The last one, *WHAD*, supports tools for different wireless protocols, including BLE. The BLE tools can scan for devices, perform Service Discovery; read, write, and subscribe to characteristics; sniff traffic; crack the Legacy Pairing handshake; filter packets by type or BDAs; relay, replay, and inject packets; and spoof advertisements and GATT servers [36].

BLE-Replay supports sniffing, modifying, and replaying BLE messages. For replaying data, *BLE-Replay* takes as input the content of the Bluetooth Host Controller Interface (HCI) log of a mobile device and extracts all write packets sent from the mobile phone. The extracted data can be modified or replayed in an arbitrary order. Moreover, *BLE-Replay* can be used for byte-level fuzzing [57].

Four frameworks contain a broad range of tools to conduct different BLE attacks.

BtleJack allows sniffing, jamming, and hijacking BLE devices [88].

GATTacker uses two Bluetooth adapters serving as Central and Peripheral, enabling attacks such as BDAs and advertisement spoofing, passive eavesdropping, MitM attacks to intercept and modify packets, command injection or replay, Service Discovery, read and write operations, and fuzzing [37].

BLEsuite CLI is based on the Python package *BLE-Suite* and allows scanning for devices, conducting Service Discovery and a so-called *smart scan*. The *smart scan* is similar to the Service Discovery, but instead of reading the characteristic values, the characteristic descriptor values are read to generate a populated BLE device instance. In addition to that, BDAs spoofing and reading, writing, and subscribing to characteristics, in either synchronous or asynchronous mode, is possible [47].

Sivakumaran and Alis [54] developed an open-source *ATT-Profiler* to identify the lowest security level required for accessing the value of a characteristic of a BLE device. Additionally, the Profiler can detect whether a static PIN code was used during authentication by comparing it against a dictionary of commonly used PIN codes.

Zendehdel et al. [26] developed and tested a framework named *SAFEMed*, based on the Penetration Testing Execution Standard (PTES) [89] guideline stages, to semi-automatically identify known and unknown vulnerabilities in the ecosystem of Wearable Health Monitoring Devices (WHMDs). After extracting device-specific technical information, the framework conducts a CVE, NVD, and CWE search to identify vulnerabilities and weaknesses in the examined device. In addition, BLE communication is tested to determine whether Service Discovery can be performed and whether the device is susceptible to eavesdropping, packet tampering, relay and replay, fuzzing and advertisement spoofing.

3) TOOLS AND FRAMEWORKS RUNNING ON DIFFERENT HARDWARE OR OSES

This subsection presents tools and frameworks that can run on other OSES, in addition to or instead of Linux, or use an Software Defined Radio (SDR).

bettercap runs on different OSES and offers penetration testing tools for multiple wireless communication protocols used by IoT devices. For BLE, a user can conduct Service Discovery and read or write to characteristics of a BLE device [46]. However, the BLE operations work only on Linux [90].

The framework *Snout* offers functionalities for Bluetooth, covering BDAs collection, device tracking, and analysis of traffic information such as message frequency or uptime. On top of that, it provides tools for device analysis, i.e. message analysis to extract information such as device vendor, OSES, model, and other details, mostly for Apple devices [29]. *Snout* uses *GNU Radio* as an SDR.

BTLEmap [30] supports Service Discovery, advertisement dissection, and proximity estimation based on the RSSI value. *BTLEmap* only runs on iOS and macOS.

Nordic Semiconductor provides the application *nRF Connect Bluetooth Low Energy*, developed to test self-developed

BLE devices, but can also be used to gather more information about a remote device. For example, it can scan for devices, perform Service Discovery, and display information about the advertising or connection state of the peripheral. Moreover, it allows configuring the Central, including changing its advertising information, pairing and connection parameter and services [40]. It can be used with Nordic Semiconductor's nRF development kits or dongles.

Wireshark is a network protocol analyzer that can be used to inspect captured traffic in greater depth across different protocols, for example Wi-Fi or Bluetooth. Traffic can be captured live or opened from a pcap file. Captured traffic can also be saved as a pcap file for later analysis [91, 1.1.5.2.2.].

D. MOBILE SOLUTIONS

This subsection presents mobile apps and phone settings that support the security testing of BLE devices.

The apps *nRF Connect for Mobile* and *LightBlue* (both for Android and iOS) allow scanning for BLE devices, performing Service Discovery, spoofing advertisement data (*nRF Connect for Mobile* only) and setting up a fake GATT Server (iOS only for *LightBlue*) [39], [49], [92], [93].

Another helpful tool is Android's *Bluetooth HCI Snooper Log*. It captures all the Bluetooth HCI packets exchanged between the phone and a peripheral and can export them for further inspection with *Wireshark* [94].

VI. TEST AND EVALUATION

In this section, we introduce the test devices used to evaluate the tools, frameworks and Linux packages and utilities described in the previous section, followed by an in-depth evaluation. For each tool, framework, package and utility, we briefly summarize the attacks it supports, along with a detailed summary of the results. In the remainder of this paper, we use the term "tools and frameworks" to also refer to Linux packages and utilities.

A. TEST DEVICES

To evaluate the tools and frameworks introduced in Section V, we employed a range of devices and operating systems.

Our test devices were either medically certified or consumer devices able to monitor physiological parameters. Using consumer devices for our evaluation was of particular interest due to their widespread use and their potential role in clinical studies [95]. The medically certified devices was the glucose meter *One Touch Ultra Plus Reflect*,¹ denoted as *OneTouch R113* in the following. The consumer devices included the *Muse S* headband,² an EEG-powered meditation and sleep headband, and two smart rings (*AMZMAX Smart Ring*³ and *COLMI R02 Smart Ring*⁴), denoted as *Smart Ring*

¹<https://www.onetouch.de/produkte/blutzuckermessgeraete/onetouchreflect>

²<https://choosemuse.com/products/muse-s-gen-2>

³<https://de.aliexpress.com/i/1005007163218252.html?gatewayAdapt=glo2deu>

⁴<https://www.colmi.info/products/colmi-r02-smart-ring>



FIGURE 2. The four test devices. Own pictures.

I and *Smart Ring II*, respectively, in the rest of the work. The selected devices reflect representative IoMT-oriented devices but cannot cover the full diversity of BLE implementations.

Pictures of each device can be found in Fig. 2

In addition to that, we used a live image of Kali Linux (version 2025.1a) running from a USB stick and version 2024.4 running in a Virtual Machine (VM), both on a Dell Latitude 5330. When using the VM, we used the BT-8500 Bluetooth dongle from Edimax. We decided to use Kali Linux as the Linux distribution for testing, since Kali Linux is designed for digital forensics and penetration testing. Other Linux distributions might work as well. For testing the mobile apps, we used a Fairphone 4 with Android version 13, a Lenovo Tab tablet with Android version 15 and an iPad running iOS 14.4. Moreover, we had a MacBook Air with macOS 14.6, and a Dell Latitude 5330 running Windows 11.

In the subsequent sections, we describe our test setup for each of the tools and frameworks we tested, followed directly by their evaluation. The commands used in each attack for all tools and frameworks and additional material can be found on Github: <https://github.com/HannahGress/BLE-Security-Testing-A-Survey-of-Tools-and-Frameworks>. We tested each tool and framework on all four test devices whenever possible.

B. HARDWARE TOOLS FOR PENETRATION TESTING AND VULNERABILITY ASSESSMENT

We describe the test setup and evaluation of the *Ubertooth One* in Section I when assessing its appropriate Linux package *ubertooth*. We evaluate the *nRF52840 Dongle*

as a representative for the Nordic Semiconductor family in Section I, since its firmware is closed-source and therefore fits better in the subsequent section. We excluded Adafruit's *Bluefruit LE Sniffer*, as its underlying chip is discontinued and will soon be obsolete. We used the same testing setup for both of the remaining HW sniffers that we will outline in the respective sections.

C. CLOSED-SOURCE TOOLS AND FRAMEWORKS OR WITHOUT PUBLICLY AVAILABLE SOURCE CODE

Four out of five tools and frameworks in this category lack publicly available source code and therefore could not be tested. Consequently, only the *nRF Sniffer for Bluetooth LE* was evaluated, as its firmware is available but closed-source.

1) NRF SNIFFER FOR BLUETOOTH LE

Test Setup: We first recorded the messages exchanged between each test device and its corresponding app during data synchronization. Second, we recorded the traffic between the test device and the *nRF Connect for Mobile* app when attempting to read the value of each characteristic with the property *Read*. We also recorded the pairing handshake of the *OneTouch R113* to evaluate the Linux package *crackles* (see Section I). We exported all recordings to pcap files. In addition to that, we exported the *Bluetooth HCI Snoop Log* serving as the ground truth. We compared both recordings in *Wireshark* to verify whether the *nRF52840 Dongle* captured all packets and, if not, to determine the packet loss rate.

Evaluation The *nRF52840 Dongle* listens to all three advertisement channels simultaneously, which facilitates

capturing the connection establishment required for subsequent recordings. We successfully recorded all packets in all three cases, even under high transmission rates as observed with the *Muse S*. When inspecting the recorded pcap files in *Wireshark*, we could see that the packet source and destination were labeled according to their respective roles. In addition to that, the Universally Unique Identifiers (UUIDs) of predefined characteristics were resolved into their names. Both points support the understandability of the message exchange.

D. TOOLS AND FRAMEWORKS WITH PUBLICLY AVAILABLE SOURCE CODE

1) PACKAGES AND UTILITIES BASED ON THE LINUX BLUETOOTH STACK BLUEZ

a: *bluetoothctl* In general, the *bluetoothctl* utility offers commands for Service Discovery and operations suitable for a spoofing attack. Spoofing can include the spoofing of advertisement data and the disguise as a GATT Server or Client. In the following, we first outline how to perform Service Discovery, followed by the two variants of spoofing attacks.

I.) *Service Discovery Test Setup*: The submenu *gatt* provides commands for Service Discovery and commands to read, write, or subscribe to notifications from a characteristic. For our tests, we limited the evaluation to reading values. **Evaluation**: We could successfully perform Service Discovery on all our test devices. Reading the characteristics also worked well.

II.) *Spoofing of Advertisement Data Test Setup*: Advertisement data varies by device and may include the advertising physical channel PDU, flags, Service UUIDs, or manufacturer-specific data [25, 3.C.11, 6.B.2.3.1], [96, A.1]. The *bluetoothctl* utility provides the submenus *mgmt* and *advertise* for configuring advertisement data; both were tested. **Evaluation**: The success of advertisement spoofing depends on the data to be broadcasted. The command *add-adv* in the submenu *mgmt* did not work. Either an instance was added without broadcasting any data, or no data could be added at all, and only the available options for the command *add-adv* were returned. The commands in the submenu *advertise* worked more reliably, but several issues occurred. The command *data* failed either directly or at the end of the whole command flow. This limited spoofing attempts, since some fields can only be set with data, e.g., the Incomplete List of 32-bit Service or Service Class UUIDs (*data 0 × 04 <UUID>*), because the command *uuids* only sets the Complete List of 16-bit Service or Service Class UUIDs [52, 2.3]. The command *discoverable-timeout <time in seconds>* sets the LE Limited Discoverable flag correctly, but all the flags are only broadcasted as long as the discoverable timeout is not reached. Afterwards, the advertisement data is broadcasted without the flags. Setting the Peripheral Connection Interval Range can be done with the *interval [min] [max]* command, but the minimum value is 20.

Therefore, configuring the required interval of 15.00–30.00 ms for the *Muse S* was not possible. Since the TX Power Level is defined in the Bluetooth stack implementation, it cannot be changed in the advertisement data. The *Muse S* has a TX Power Level of 1dB, but our computer 7dB, so the value of 7dB was advertised instead of the 1dB which we specified in the spoofed advertisement data. In addition, combining commands or omitting parameters can result in errors. Using the commands *manufacturer* and *appearance* together, or executing *manufacturer* with only the *id* parameter and no *data*, both led to an error when calling *advertise on*.

III.) *Spoofing by Creating a Generic Attribute Profile Client or Server Test Setup*: Besides spoofing the advertisement data, we also tried to spoof our test devices as GATT Server. For the GATT Server spoofing, we manually added the services, characteristics and the properties our test devices expose. **Evaluation**: Adding a service or characteristic with *bluetoothctl* is possible, but tedious. Each service and characteristic must be added separately, because for each service the user must confirm that the service should be added as Primary Service and a default value must be assigned to each characteristic directly after registering. Another drawback is that the Generic Access Profile (GAP) and GATT Service (0 × 1800 and 0 × 1801, respectively [52, 3.4.1]) cannot be overwritten, preventing full recreation of a remote device. Implementing the GAP Service is mandatory for both the GATT Server and GATT Client, while implementing the GATT Service is required only for the GATT Server [25, 3.C.12, Table 12.1; 3.G.7, Table 7.1]. This explains why modifying these services is not allowed. Moreover, when there is a syntax error in a command, the command is accepted, but advertising fails.

IV.) *Spoofing by Cloning a Remote GATT Client or Server Test Setup*: An easier way to configure a GATT Client or Server with the same configuration as a remote device is to use the *clone* command in the submenu *gatt*. **Evaluation**: The *clone* command is faster and less error-prone, but it does not fully clone a device; rather, it adds the device's services and characteristics to those already present on the Bluetooth chip. However, if the remote device owner is unaware of the services the device exposes, the *clone* command is well suited for impersonation attacks. The computer which represents the attacker must remain connected to the impersonated device to present its services and characteristics to other devices. Consequently, the remote device owner can connect only to the attacker's device, facilitating the impersonation. Since the attacker remains connected to the impersonated device, read — and potentially write and subscribe operations (not tested) — can be forwarded. Reading values from characteristic descriptors did not work in our experiments. In both cases — creating or cloning a GATT Server or Client — advertisement data must be configured separately.

b: *Blue-Hydra Test Setup*: We conducted the tracking in three ways: leaving the tracking list unchanged, highlighting specific BDAs, or filtering for specific BDAs [28].

Evaluation: Device tracking worked as expected, including highlighting and exclusive scanning for specific BDAs.

c: ubertooth Test Setup: Before sniffing, we set the target address to the test device under evaluation. We then repeated the same test used for the *nRF52840 Dongle* (see Section I). We first recorded the messages exchanged between each test device and its corresponding app during data synchronization. Second, we recorded the traffic between the test device and the *nRF Connect for Mobile* app when attempting to read the value of each characteristic with the property *Read*. We also recorded the pairing handshake of the *OneTouch R113* to evaluate the Linux package *crackle* (see Section I). We exported all recordings to pcap files. In addition to that, we exported the *Bluetooth HCI Snoop Log* serving as the ground truth. We compared both recordings in *Wireshark* to verify whether the *Ubertooth One* captured all packets and, if not, to determine the packet loss rate. **Evaluation:** Since the *Ubertooth One* can only listen to one advertisement channel at a time [43], we often needed multiple attempts to capture the an ongoing connection after observing advertisement data. When comparing the exchanged packets recorded with the *Ubertooth One* and the *Bluetooth HCI Snoop Log*, we made the following observations: The *Ubertooth One* did not consistently capture all packets, and packet loss increased with higher transmission rates. We could see this very well by inspecting the recorded packets between the *Muse S* and its appropriate app. Here, only 5% of the packets were captured by the *Ubertooth One* compared to the *Bluetooth HCI Snoop Log*. In addition to that, some packets were incorrectly recorded, causing *Wireshark* to display L2CAP Fragment [BoundErrorUnreassembled packet]. Moreover, the *Ubertooth One* did not record the sender and the receiver of the packet, complicating packet attribution. We could often deduce it from the operation, e.g. we knew that we performed a read request with the *nRF Connect for Mobile* app, so the recorded read request with the *Ubertooth One* was from the app and the read response from the test device. Nevertheless, explicit identifiers such as BDAs or names would improve readability. When we inspected a BLE packet in depth, we could see the Central's and Peripheral's BDAs, but they did not allow reliable packet attribution. Lastly, for each characteristic, only its handle was shown, even for predefined characteristics with assigned names, such as the Device Name. That being said, identifying the standardized services and characteristics is harder compared to the *Bluetooth HCI Snoop Log* where *Wireshark* can add the human readable definition for each of them automatically.

d: crackle Test Setup: We were only able to test *crackle* with the *One Touch R113*, the sole device that performs pairing. Moreover, it supports only Legacy Pairing, which *crackle* requires. The *nRF52840 Dongle* is controlled via *Wireshark*; therefore, only the *crackle* command applies to its captured pcap files. **Evaluation:** We could neither recover the TK nor decrypt the pcap files we provided to *crackle*. It returned the error No connect packet

found. Giving up due to 1 error, even though we had recorded the `CONNECT_IND` packet.

2) TOOLS AND FRAMEWORKS ONLY RUNNING ON LINUX

a: EXPLIoT Test Setup: We performed Service Discovery and tried to fuzz characteristics with the property *Write*. **Evaluation:** Service Discovery worked without issues. However, a drawback in the output format is that services are first listed with their handle ranges, followed by characteristics with their handles, making it necessary to manually map characteristics to services. Determining the mapping requires manually comparing each characteristic's handle with the corresponding service's handle range. Fuzzing characteristics was also successful. We deliberately selected a low iteration rate — that is, how frequently values were sent — and targeted characteristics with known purposes (e.g., Device Name). As a result, no crashes occurred during testing.

b: HomePwn Test Setup: We performed Service Discovery and tried to read every characteristic with the property *Read*. In addition to that, we tested the *MAC spoofing module* (BDAs spoofing). Since sniffing only monitors general BLE traffic in the surroundings and cannot target a specific device (address), it was not tested, as it is neither useful nor suitable for our testing approach. **Evaluation:** Service Discovery lists only characteristics, not services. *HomePwn* lists the handles of the characteristics, but since no services and their handle ranges are displayed, the handles cannot be matched to services, as is the case with *EXPLIoT* (see II). A useful feature of *HomePwn* is that the values of the characteristics are directly read and displayed if they possess the property *Read*. However, directly reading characteristics also has limitations. For the *Smart Ring II* and the *Muse S*, the device names could not be read or were read incorrectly (b' ' and nothing as output, respectively). Additionally, hex-encoded values cannot be properly decoded, e.g., Appearance, System ID, PnP ID, Peripheral Preferred Connection Parameter, or IEEE 11073-20601 Regulatory Certification Data List. The values were either Couldn't read or empty. Spoofing failed, as the required module could not be loaded.

c: SecuWear Test Setup and Evaluation: The framework *SecuWear* provides three methods to monitor and record Bluetooth traffic. The first method is the hardware sniffer *Ubertooth One*; the other two methods are implemented in a custom-written app. The app can extract the *Bluetooth HCI Snoop Log* from the phone and record events performed by the Device Under Test (DUT) *MetaWear*, e.g., temperature readings from a sensor. The app dates from 2017, so we would have had to make extensive changes to the source code to possibly get it running on our test phone with Android 13; however, we did not attempt this due to its age. Sniffing with the *Ubertooth One* is already covered in Section I. Therefore, we decided not to continue testing *SecuWear* and mark it as no longer usable.

d: WHAD Test Setup: Our first two test cases were Service Discovery and reading characteristics with the property *Read*. As a third test, we conducted advertisement spoofing

manually and automatically. For the automatic advertisement spoofing, we first had to sniff and dump advertisement data from our test devices into pcap files to replay them. We also replayed modified advertisement data. To do so, we changed the BDAs of the recorded advertisement data before broadcasting. Our fourth test was to set up a GATT Server to impersonate our test devices. The fifth test was a full spoofing attack by cloning the advertisement data and all services and characteristics of our test devices. Here, our fake device was connected to the spoofed device and advertised itself as such. We connected our fake device to our mobile phone, which ran the *nRF Connect for Mobile* app. All commands issued in the *nRF Connect for Mobile* app were routed through the fake device to the spoofed device and back. The same attack can be performed using two commands: `wble-proxy` and `wble-spawn` that we both evaluated. As a sixth test, we recorded the Legacy Pairing handshake of the *OneTouch 113* to later compute the TK and Short Term Key (STK) to encrypt the subsequent communication. Our last test was the injection of data during connection setup. In this test, we tried to read and write data.

Besides the commands used in our tests, *WHAD* provides additional commands in its documentation [36] and the functions provided in the source code [97] allow developing additional (attack) tools. For all our tests, we used the *nRF52840 Dongle* by Nordic Semiconductor with the corresponding *WHAD* firmware. **Evaluation:** When conducting Service Discovery, we found that the *Signed Write* property of a characteristic was either not shown or unsupported. We also noted that the database of defined services is up to date, because the service and characteristics of the service `0xFE8D` (Interaxon Inc. [52, 3.11]) exposed by the *Muse S* were resolved into human-readable labels, unlike in the *nRF Connect for Mobile* app, for example. The app is based on Bluetooth version 4.1.

Service discovery on the *Smart Ring II* failed; we always received an `ReadNotPermittedError`.

Reading characteristics was successful and efficient, as specifying handle values instead of UUIDs simplified input and slightly accelerated the workflow. However, certain characteristics encoded as hex values —such as Appearance and Peripheral Preferred Connection Parameter —were not decoded correctly. Reading values from the *Smart Ring II* was possible by guessing handle values. The successful return of a human-readable value (e.g., Manufacturer Name) implicitly confirmed read access to the handle, despite lacking prior Service Discovery.

Manual advertisement spoofing is limited. The BDAs cannot be changed — only the Complete Local Name, Shortened Local Name, and Manufacturer data. The Device type (LE only), Advertising type (Legacy), and Flags (LE General Discoverable) are inherited from the *nRF52840 Dongle*.

The values are correct, except that our test devices additionally include the Flag `BR\ EDR Not Supported`, and the *OneTouch R113* is limited discoverable, not general discoverable (LE Limited Discoverable). Setting the Manufacturer

data (= Manufacturer ID and data) works as expected, provided that both the Manufacturer ID and data are set. The *Smart Ring II* provides only the Manufacturer ID, but setting it without associated data was not possible.

Automatic advertisement spoofing also has limitations. We successfully sniffed advertisement data from three of four test devices. We could not sniff any advertisement packets from the *Smart Ring I*, even after scanning all three advertisement channels sequentially (only one channel can be monitored at a time). However, we detected it using the standard `scan` command. Replaying the advertisement data did not work; we could not see any of the fake advertisements on our mobile phone, even though the sent advertisement packets were printed to the console, as expected per the documentation [98]. The same held for modified advertisement data. We saw from the console output that the BDAs was changed, but we were unable to capture any advertisements.

Setting up a GATT Server was successful with some limitations. The characteristic properties *Signed Write* and *Write Without Response* cannot be set [99, 1.955 ff], but the *Write Without Response* property can unofficially be set using `writecmd`. The BDAs cannot be changed and remains on `11:22:33:44:55:66`. Lastly, Characteristic Descriptors that are not automatically set cannot be set manually. For example, the *Smart Ring II* has the characteristic `0 × 2A26` (Firmware Revision String) in the service `0 × 18A0` (Device Information) which requires two Descriptors (possibly due to an implementation issue), but we had no possibility to set them. Assigning default values to characteristics is also possible. When a remote device reads or subscribes to the characteristic, the appropriate value is returned.

Cloning the test devices using `wble-proxy` was partially successful. The advertisement data was copied correctly, and the BDAs of the *nRF52840 Dongle* was changed to match that of the impersonated device. Except for the *Smart Ring I*, all names were cloned and advertised. However, as soon as we connected to the fake test devices, we noticed different behaviors. We could establish a connection between our fake *Smart Ring II*, *Muse S*, and *OneTouch R113* and the *nRF Connect for Mobile* app, but no services or characteristics were displayed. By inspecting the recorded pcap files, we could see that the Service Discovery immediately after connection setup was not performed or only partially completed, and only a few packets were exchanged. For the *Smart Ring II* we again received a `ReadNotPermittedError`.

Cloning the test devices using `wble-spawn` was fully successful only for the *Smart Ring I*. Extracting services and characteristics into JavaScript Object Notation (JSON) files, the first step in impersonation, succeeded for three of four devices. The *Smart Ring II* returned `[!] ATT error: read operation not allowed`. The second step — spawning a fake BLE device — succeeded only for the *Smart Ring I*. We successfully performed read operations with the *nRF Connect for Mobile* app on the *Smart Ring I* while the traffic was routed through our fake device, as observed live in *Wireshark*. We could set up a connection from either the

Muse S or the *OneTouch R113* to the fake device. However, when we connected the *nRF Connect for Mobile* app to the fake device, no services and characteristics were displayed as would normally be expected. As a result, routing the BLE traffic was not possible.

Sniffing the Legacy Pairing handshake succeeded. Brute-forcing the TK and calculating the STK did too, but decryption of the subsequent data failed. During the decryption process, each packet was printed on the console with some metadata, and it appeared that *WHAD* did not recognize that some of the packets were encrypted. This likely explains why decryption of the packets did not work.

Packet injection failed. According to the console output, we were able to send read and write commands, but writing was only possible in text format —not in hexadecimal; the latter returned an error. Since the write operation on the Device Name of the *OneTouch R113* is permission-free, the failure likely stems from *WHAD*.

e: **BLE-Replay Test Setup and Evaluation:** *BLE-Replay* uses the `btsnoop_hci.log` from an Android device to extract and replay write messages. As modern smartphones save the internal HCI communication in `.cfa` files, we could not test this tool anymore and consider it deprecated.

f: **BtleJack Test Setup and Evaluation:** *BtleJack* was developed for the discontinued nRF51822 System on a Chip (SoC) and has not been updated to support newer chips, e.g., the nRF52840 [100], [101]. Consequently, we decided not to purchase and evaluate the outdated hardware and to mark *BtleJack* as deprecated.

g: **GATTacker Test Setup and Evaluation:** We were able to set up the tool successfully, however, scanning for devices failed with a previously reported error [102]. As no fix was available at the time of writing, we did not investigate the issue further. Therefore, we consider *GATTacker* no longer usable.

h: **BLESuite CLI Test Setup:** For our test, we performed Service Discovery, a *smart scan* (i.e., reading the device's descriptors to generate a BLE device class instance), BDAs spoofing and tried to read the characteristics. **Evaluation:** Connection establishment — required for all tests except for BDAs spoofing — failed with an error. As a result, we attempted only BDAs spoofing, but our Bluetooth dongle did not support it. Therefore, we classify this framework as deprecated.

i: **Attribute Profile (ATT)-Profiler Test Setup and Evaluation:** We successfully installed the *ATT-Profiler*, however, when we started scanning for nearby devices, the program exited with the error `Have not received scanning notification from HCI interface yet. Please make sure a BLE interface is available and try again.`, even though our HCI interface was recognized by the OSes. Therefore, we classify the *ATT-Profiler* as no longer usable.

j: **SAFEMed Test Setup and Evaluation:** Since this work is based primarily on the Python packages *BLESuite* and *GATTacker* which we already evaluated as no longer

usable, we also consider the work of Zendehele et al. deprecated.

E. TOOLS AND FRAMEWORKS RUNNING ON DIFFERENT HARDWARE OR OPERATING SYSTEMS

a: **bettercap Test Setup:** We connected to each test device and performed Service Discovery. **Evaluation:** Service Discovery succeeded. *bettercap* directly reads and displays the values of the characteristics with the property *Read*. In a few cases, a value encoded in hexadecimal was incorrectly decoded — for example System ID or IEEE 11073-20601 Regulatory Certification Data List — or not properly converted. The incorrect conversion occurred with the Peripheral Preferred Connection Parameters. The converted value were displayed as `Connection Interval: 65535 -> 65535` instead of `Connection Interval: no specific minimum - no specific maximum`. If no specific minimum or maximum is given, the octet that define these values is set to `0xFFFF` which equals 65535 [25, 3.C.12.3]). Especially for the *Muse S*, we needed several attempts to connect, but the other three devices connected on the first try.

b: **snout Test Setup and Evaluation:** We were unable to install a compatible GNU Radio version (3.7.x) for *snout* on our system, which is a prerequisite for running the framework. Therefore, we consider *snout* deprecated.

c: **BTLEMap Test Setup:** We tested all three available functionalities: scanning for devices, displaying them in the *Proximity View*, and the *RSSI Graph*. **Evaluation:** All functionalities *BTLEMap* offers work as expected; however, we discovered minor issues. When scanning for devices, *BTLEMap* often only displays a UUID instead of the device's name. However, for all our test devices, their advertised name was shown alongside the UUID assigned by *BTLEMap*. The *Proximity View* appeared cluttered with nearby devices, but their number can be limited by setting a threshold via the implemented RSSI filter. A strength of the *RSSI Graph* is the ability to assign colors to the devices and their lines in the graph making them distinguishable.

One overall drawback we observed with *BTLEMap* is that the list of devices is not updated in real time.

d: **nRF Connect for Desktop - Bluetooth Low Energy Test Setup:** The *Bluetooth Low Energy* app offers three functions of interest: Service Discovery, configuration of a GATT Server, and setup of advertisement data. The latter two can be used for a spoofing attack. **Evaluation:** Service Discovery worked well. In addition, the app displays the value of a characteristic when it is able to read it. Configuring a GATT Server to impersonate the test devices did not work as expected; we received various errors or could only register a subset of the services and characteristics. Advertisement spoofing succeeded, but only if the advertisement data did not exceed 31 bytes — the maximum length of a legacy advertisement data packet [25, 4.E.7.8.7]. The advertisement data of the *Smart Ring I*, *Muse S* and *OneTouch R113* is longer, so it was impossible for us to fully clone the

advertisement data. However, we were able to customize the advertising data as needed, since the advertising setup permits entering any AD type, and their appropriate value. We obtained the values either from the resolved or raw advertisement data of our test devices.

e: Wireshark Test Setup and Evaluation: We used *Wireshark* solely to inspect our recorded sniffing files. Therefore, we did not evaluate *Wireshark* itself; rather, we used it as a tool to evaluate the reliability of our sniffers.

F. MOBILE SOLUTIONS

1) NRF CONNECT FOR MOBILE

Test Setup: We evaluated the app on an Android phone, because only the Android app supports all the functions that we intended to assess [39]. First, we performed Service Discovery. Second, we attempted to read each characteristic with the property *Read*. Third, we used the clone option of the *nRF Connect for Mobile* app, which allows cloning the advertisement data of a remote device. Lastly, we used the *Configure GATT server* option in the main menu and the *New advertising packet* option in the *Advertiser* tab to manually create a fully spoofed device. For both spoofing tests, we compared the advertising data and the exposed services and characteristics of the fake and spoofed device using the *nRF Connect for Mobile* app running on another device. **Evaluation:** Service Discovery and reading characteristics succeeded. Fake GATT Server configuration failed. The server was set up successfully, but its services and characteristics were not exposed. In contrast, the *nRF Connect for Mobile* includes a built-in sample configuration of a GATT Server whose services and characteristics could be exposed. But even in the sample configuration, the Generic Access and Generic Attribute Profile services were sourced from the phone and the others from the sample configuration.

Cloning advertisement data has limitations. As the *nRF Connect for Mobile* app states in a popup window, the Android Application Programming Interface (API) does not allow setting the Complete Local Name, TX Power, or Flags; these fields default to the phone's values. The Appearance cannot be set or cloned. In addition, an Android bug allows setting multiple instances of the Service Data and Manufacturer Data. We further encountered that the Peripheral Connection Interval Range and BDAs are also not cloned; the latter is always random and changes every 15 minutes and every time we stop and start advertising again. If a device advertises an Incomplete list of 16-bit Service UUIDs, as is the case for the *MuseS*, the list is changed into a Complete List of 16-bit Service UUIDs and broadcasted as such. Furthermore, the device type (LE, Classic or both) is also taken from the phone, even if it is shown differently in the app. The app displayed for all our test devices LE only as device type, but LE and CLASSIC was actually advertised. So what is cloned and advertised from the test device are the advertising type (Legacy for all test devices), the Service Data and the Manufacturer Data. Even if the Complete Local Name is derived from the phone, we can

TABLE 2. Comparison of the Built-In or Manually Cloning Function in the *nRF Connect for Mobile* App for Advertising Data. The Advertisement Types are Taken From the Test Devices and Do Not Cover All Possible Advertisement Types. ✓ Indicates That the Value From the Impersonated Device Can be Successfully Cloned, ✗ That the Value Was Either Taken From the Phone or the Type Could Not be Added to the Fake Advertisement Data.

Advertisement Type	Cloned	Manually
Device Type	✗	✗
Advertising type	✓	✓
Complete Local Name	✓ ^a	✓ ^a
Appearance	✗	✗ ^b
Flags	✗	✗
Complete List of 16-bit Service UUIDs	✓	✓
Incomplete List of 16-bit Service UUIDs	✗	✗
Service Data	✓	✓
Peripheral Connection Interval Range	✗	✗
TX Power Level	✗	✗
Manufacturer Data	✓	✓

change it in the advertisement settings of the *nRF Connect for Mobile* app.

Manual crafting of fake advertisements is similarly limited. The Flags, the Complete Local Name and the TX Power are taken from the phone, when they are selected for the advertisement data. As with the cloned advertisements, the Complete Local Name is changeable in the advertising settings. Likewise, the Incomplete List of 16-bit Service UUIDs cannot be set, only the Complete List of 16-bit Service UUIDs. What can freely be set is the Complete List of 16-bit Service UUIDs, the List of 16-bit Service Solicitation UUID, the Service Data and Manufacturer Data. However, if the Manufacturer Data should be set, which consists of manufacturer ID and the data, both fields need to have a value. This was particularly problematic for the *Smart Ring II*, because it officially broadcasts only the manufacturer ID and no additional data. In addition to the above-mentioned, the manually crafted advertisement data allows changing the advertisement type to Extended Advertising, if necessary.

Since all of our test devices use legacy advertisement, the broadcasted data is limited to 31 bytes [25, 4.E.7.8.7]. Therefore, we were not always able to spoof the full advertisement data, because the app threw a Legacy advertisement data is too big error, even if we could successfully clone and advertise the same data.

Table 2 summarizes our findings by comparing the different advertising types of the built-in cloning function against our manually cloned data regarding their success — i.e., whether we could change the value of each advertising type of our test devices to successfully impersonate it. It can clearly be seen that not every advertising type value can be changed. However, since the whole advertising data is normally hidden for a user when scanning for devices on their mobile phone, changing only the Complete Local Name already helps to create a more plausible spoofing attack.

^aAdjustable in the advertising settings.

^bNo option available to set the Appearance.

2) LIGHBLUE

Test Setup: We conducted our test on an iPad, as only the iOS/macOS version of the *LightBlue* app supports device spoofing. This can be achieved either by manually configuring a GATT Server or by using the clone option of the app to create a fake GATT Server. In addition, we tested the Service Discovery function. **Evaluation:** The Service Discovery function worked only partially. If a device exposes the GAP or GATT Service, they are not displayed in the *LightBlue* app. However, if a characteristic is readable without security permissions, the app automatically displays its value.

Manually creating a fake GATT Server is also limited. The GAP, GATT and Device Information Service cannot be added or modified.

Similar to manual creation of a GATT Server is the cloning of our test devices. When the test device has the GAP or GATT Service, these services are not added. When the device includes the Device Information Service, cloning fails with the error (Disallowed service uuid: 180A) and the device cannot be advertised. In addition, once we had changed the name of the GATT Server in the app, the advertised name remained unchanged, even after renaming the GATT Server, impersonating another test device, or reinstalling the app.

In both cases, when connecting our mobile phone to the fake GATT Server, we observed that all default iPad services were also exposed, in addition to those we cloned or created manually. If a characteristic has the property *Write* or *Write Without Response*, the app automatically adds the property *Extended Property* along with the corresponding Characteristic Descriptor *Characteristic Extended Properties*. In addition, the name of the fake GATT Server must have at least 15 characters, otherwise the fake device is not advertised. Additionally, this name only appears in the advertisement data; the *Device Name* in the GAP Service remains unchanged from the iPad's system settings.

We believe that the GAP, GATT and Device Information Service are not the only services that cause problems when cloning or manually creating a (fake) GATT Server. However, we are convinced that most of the issues we encountered are simply bugs that can be fixed in the future. Therefore, we reported these issues to the developers.

All in all, the *LightBlue* app can partly be used for a spoofing attack. Even though only a few services can be added and the advertisement data or BDAs cannot be modified, a user can still be spoofed, as the advertised name can still be changed, which is likely the only attribute visible to the user. However, once the spoofed device attempts to communicate with the fake GATT Server, issues arise, since the fake GATT Server does not respond as expected, potentially raising suspicion.

3) BLUETOOTH HCI SNOOP LOG

Test Setup and Evaluation: We used the *Bluetooth HCI Snoop Log* as the ground truth for all of the sniffing tests (*Ubertooth One* and *nRF52840 Dongle*), as it records all

TABLE 3. Overview of Tool and Framework Categories and Their Counts.

Category	Count
HW Sniffer	3
Closed-source / no publicly available code	5
Linux utilities and packages	5
Running on Linux	10
Running on different OSes	5
Mobile solutions	3
Total	32

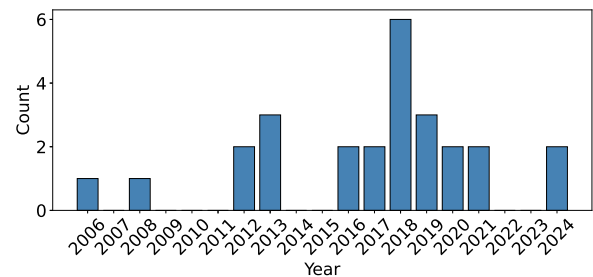


FIGURE 3. First release dates of the HW sniffers, tools, and frameworks we considered, if an official release date was available. For the two apps, we used the date of their first release—both were Android versions.

incoming and outgoing Bluetooth packets. Thus, we did not evaluate the *Bluetooth HCI Snoop Log*, we used it as a reference to assess the reliability of our sniffers.

VII. DISCUSSION

A. TOOLS AND FRAMEWORKS - OVERALL OBSERVATIONS

For our work, we identified three HW sniffers, five tools or frameworks that are either closed-source or without publicly available code, four Linux packages or utilities, ten open-source tools or frameworks running solely on Linux, five running on different or multiple OSes and three mobile solutions as summarized in Table 3. Table 4 provides a more detailed overview of all tools and frameworks including their publication or initial release date, first and last commits (if applicable), and code availability status, which forms the basis for parts of our subsequent analysis. Without publicly available code means that there is either no repository from which we can get the tool or framework or it is closed source, but we can still install the application.

Fig. 4 shows the development time for each of the tools and frameworks we could potentially test. The development time is defined as the period from the first release or commit to the latest update. The figure includes the date of the paper publication or first release, if applicable. For those with a publication or first release date, we show their distribution in Fig. 3. For this statistic, we excluded the HW sniffers and used the earlier of the Android or iOS release dates for mobile apps, which in both cases was the Android release. The figure reveals that the tools and frameworks were first developed in

TABLE 4. Overview of Bluetooth Tools and Frameworks.

Category	Name	Paper/Rel.	1st com.	Last upd.	Closed	Open
HW Sniffer	Ubertooth One [104]	2012	2010	2020		✗
	Bluefruit LE Sniffer [77]	2014	n.a.	2022	✗	
	Nordic Sniffer nRF52840 Dongle ^a [105]	2013	n.a.	2024		✗
No publicly available code	Siboni <i>et al.</i> [27]	2019	n.a.	n.a.	✗	
	<i>BLE Security Testing Framework</i> [34]	2018	n.a.	n.a.	✗	
	<i>BLEBerry</i> [33]	2024	n.a.	n.a.	✗	
	Garbelini <i>et al.</i> [51]	2020	n.a.	n.a.	✗	
	nRF Sniffer for BLE ^a [81]	2018	n.a.	2024	✗	
Linux packages and utilities	bluetoothctl	n.a.	2012	2024		✗
	blue-hydra [28]	2016	2016	2024		✗
	ubertooth [106]	2012	2010	2023		✗
	crackle [44]	2013	2013	2020		✗
Running on Linux	EXPLIoT [48]	2021	2018	2025		✗
	HomePwn [35]	n.a.	2019	2021		✗
	SecuWear [42]	2019	2018	2019		✗
	WHAD [36]	2024	2022	2025		✗
	BLE-Replay [57]	n.a.	2016	2016		✗
	BtleJack [88]	2018	2018	2023		✗
	GATTacker [37]	2016	2016	2018		✗
	BLESuite CLI [47]	n.a.	2016	2016		✗
	ATT-Profiler [54]	2018	2017	2021		✗
	SAFEMed [26]	2021	2019	2020		✗
Running on different OSes	bettercap [46]	2018	2017	2025		✗
	Snout [29]	2019	2019	2019		✗
	BTLEmap [30]	2020	2020	2024		✗
	Wireshark [107]	2006	1998	2025		✗
	nRF Connect BLE [40]	2017	2015	2025		✗
Mobile solutions	nRF Connect for Mobile [39]	2013 ^b /2023 ^c	n.a.	2024 ^b /2025 ^d	✗	
	LightBlue	2017 ^b /2020 ^e	n.a.	2025 ^b /2025 ^d	✗	
	Bluetooth HCI Snoop Log [94]	2008 ^f	n.a.	2025 ^g	✗	

the late 2000s, starting with the *Bluetooth HCI Snoop Log*, released in 2008 alongside the first Android version [103]. The release peak was in 2018.

Going back to Fig. 4, we can see that 14 tools and frameworks are still under development. We consider tools under development if the last commit occurred in 2024 or 2025. In continuation of this, we examined whether tools or frameworks linked to publications were maintained or updated after publication. We consider a tool or framework maintained if it was further developed after the related paper was published or, for repository-only tools and frameworks, if its last commit was in 2024 or 2025. We identified seven such

tools or frameworks and marked them with an asterisk in the figure. It can be seen that four tools or frameworks were maintained beyond publication, while development of the other three ceased at or before publication. It is encouraging to see that nearly half of the paper-based tools remained active after publication. We additionally compared which tools and frameworks are still usable regardless of whether development continued beyond release or publication. We define usable as meaning that we could successfully install the tool or framework and that it did not fail at runtime. This does not imply that every attack the framework claimed to support could be launched successfully on our test devices. We highlighted these still usable tools and frameworks in bold. We observe that six of eight repository-only tools or frameworks remain functional, whereas only one of seven paper-based tools or frameworks does. In addition, all except one repository-only tool or framework is still maintained with the latest commit in 2025, explaining their continued functionality. Several factors explain why many tools — especially those resulting from publications — are no longer functional: OSes changes making the functionalities needed for the tool or framework to run unavailable (*SecuWear*, *BLE-Replay*), outdated hardware (*BtleJack*), errors during code

^aThe firmware development code for Nordic Semiconductor devices, including the nRF52840 Dongle, is open source, but the specific sniffing firmware is not.

^bAndroid.

^ciOS; version 2.6.5 (2023) is the earliest listed in the version history [108].

^diOS.

^eiOS; version 4.0.0 (2020) is the earliest listed in the version history [92].

^fAndroid version 1 was released in 2008 [107]. The Bluetooth HCI Snoop Log can be enabled via the developer options, which appear to have been available since the initial release, suggesting that the Bluetooth HCI Snoop Log was likely available from the beginning as well [109].

^gAs the Bluetooth specification evolves, we believe Google regularly updates the Bluetooth HCI Snoop Log as well.

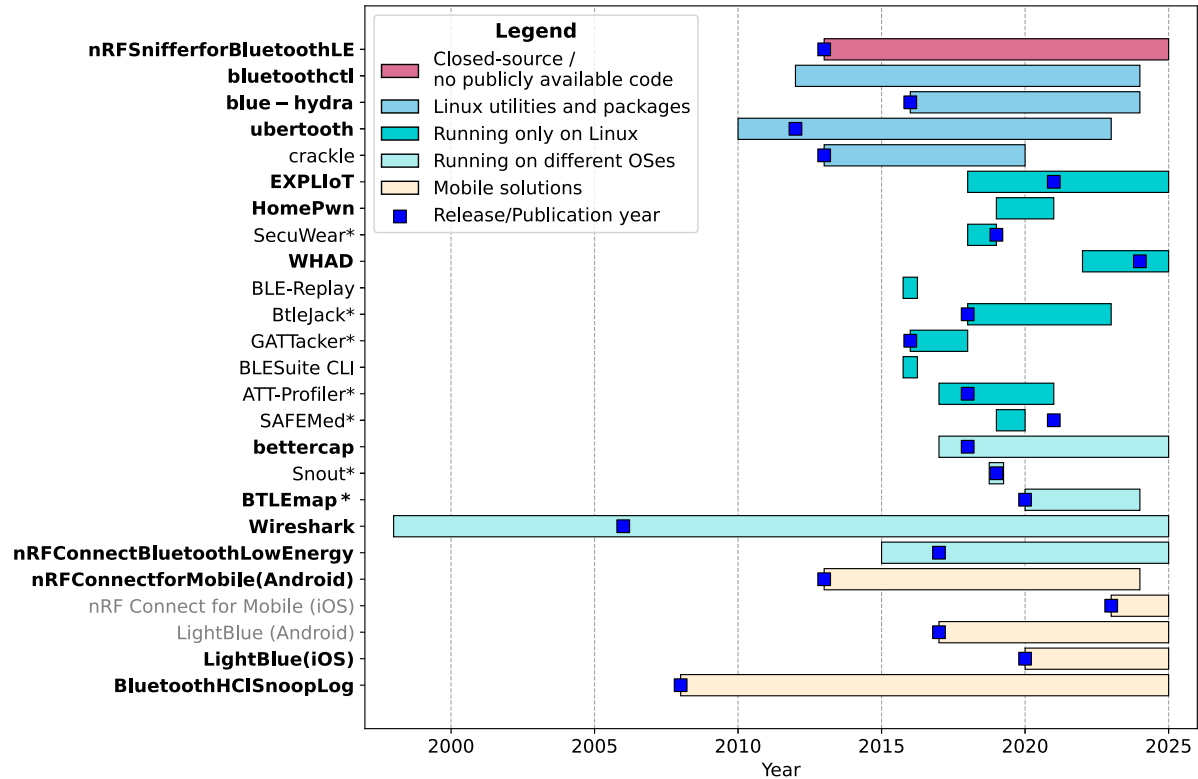


FIGURE 4. Development time for each of the tools and frameworks we could potentially test. The development time is defined as the period from the first release or commit to the latest update. The figure includes the date of the paper publication or first release, if applicable. Names marked in bold are still usable, gray names indicate the Android or iOS app version that was not tested. Names marked with an asterisk (*) are paper publications.

TABLE 5. Overview of Attack Infeasibility Due to Tool or Framework Obsolescence or Setup Incompatibility.

Tool/ Framework	Tool/Framework Obsolescence	Setup Incompatibility	Reason
ATT-Profiler		✗	HCI interface HCI not recognized despite OS detection
BLE-Replay	✗		HCI Snoop Log file format changed
BLESuiteCLI		✗	Connection setup failed
BtleJack	✗		Outdated HW; no port to newer chips
crackle		✗	Connect packet not detected despite captured
GATTacker		✗	Stuck in unknown state during BLE scan
SAFEMed	✗		Depends on deprecated <i>BLESuite</i> and <i>GATTacker</i>
SecuWear	✗		Extensive changes needed to port to modern Android
Snout	✗		Unable to install the required GNU Radio version

execution (*GATTacker*, *BLESuite CLI*, *ATT-Profiler*) or non-functioning dependencies (*SAFEMed*, *snout*). In summary, active maintenance (i.e., a last commit in 2024 or 2025) strongly correlates with continued functionality; unmaintained tools typically fail. We therefore encourage developers to maintain their BLE tools and frameworks to ensure long-term usability.

Of the 27 tools and frameworks (excluding the HW sniffers), 23 provide either publicly available source code or a compiled version, allowing installation and potential execution. From the remaining 23, we were unable to install four tools and frameworks running only on Linux (*SecuWear*,

BLE-Replay, *BtleJack*, *SAFEMed*) and one which runs on different OSes (*Snout*) due to outdated dependencies or unsupported hardware. From the 18 remaining, one of the Linux packages (*crackle*) and three of those running only on Linux (*GATTacker*, *BLESuite CLI*, *ATT-Profiler*) were installable, but failed at runtime. We summarize the failure reasons and whether they result from tool or framework obsolescence or setup incompatibility in Table 5. Detailed failure reasons for each tool or framework are provided in Section VI. In total, we successfully tested 15 tools and frameworks on the four test devices. We summarize and discuss our findings in the subsequent section.

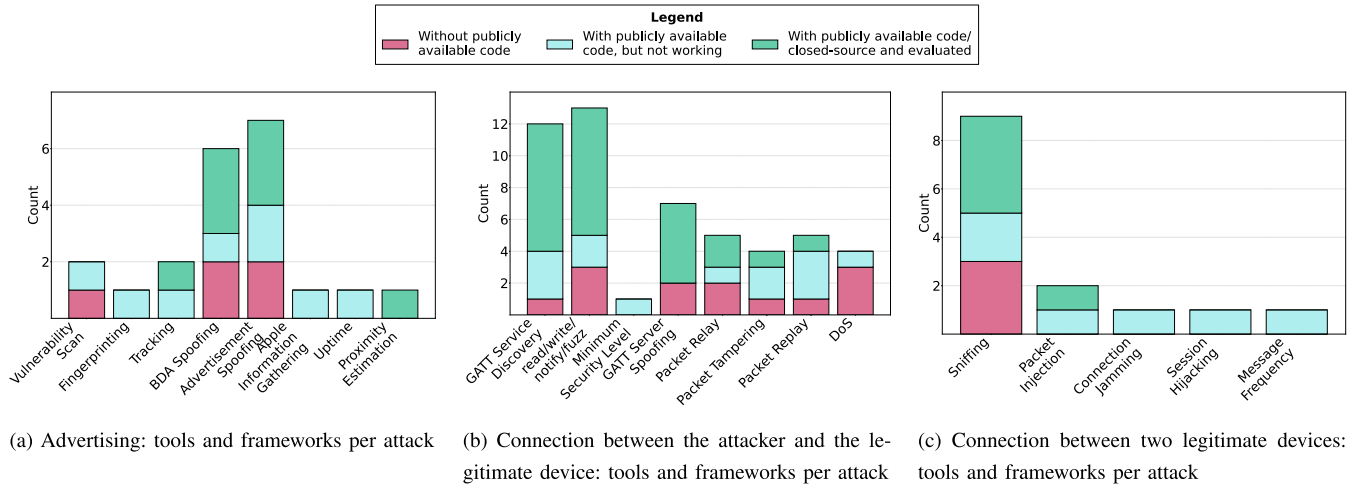


FIGURE 5. Distribution over tools and frameworks per attack. For each attack, we show the number of tools or frameworks that are able to execute it. Berry red indicates tools without publicly available code; light blue denotes open-source tools for which evaluation failed; light green shows tools (open- or closed-source) that we could evaluate on our test devices. The *Connection Establishment* phase is omitted, as all tools and frameworks in this category had publicly available code and were successfully installed and tested.

TABLE 6. Advertisement phase: BLE Prerequisites, Attacks and Results. ✓ Means That the Attack Was Fully or Almost Fully Possible, ~ That it Was Partially Possible and ✗, That the Attack Did Not Work.

Tool/ Framework	Device info. gathering	BDA/static identifier collection		RSSI								
	Vuln. Scan	Finger- printing	Tracking	BDA Spoof.	Adv. Data Spoof.	Apple Info. Gather.	Uptime	Prox. Estim.	Smart Ring I	Smart Ring II	Muse S	OneTouch R113
bluetoothctl				✗	✗				~	~	~	~
blue-hydra			✗						✓	✓	✓	✓
WHAD				✗					~	~	~	~
(manually)					✗				~	~	~	~
(wble-dump & wble-replay)					✗				✗	✗	✗	✗
(wble-proxy)					✗				✓	✗	✓	✓
(wble-spawn)					✗				✓	✗	✓	✓
BTLEmap								✗	✓	✓	✓	✓
nRF Connect Desktop				✗					✓ ^h	✓ ^h	✓ ^h	✓ ^h
nRF Connect Mobile												
(manually)					✗				~	~	~	~
(automatically)					✗				~	~	~	~

B. TOOLS AND FRAMEWORKS - OBSERVATION PER PHASE

As outlined in Section IV, we classified the possible BLE threats into four phases, (1) **advertisement**, (2) **connection establishment between two legitimate devices**, (3) **connection between an attacker and a legitimate device** and (4) **connection between two legitimate devices**. In the following, we will have another look on each phase and present the tools and frameworks used to conduct the attacks in each phase along with the results of launching each attack on our test devices.

In Table 6, we present the attacks feasible in the **advertising** phase. Here, we can clearly see that the functional tools and frameworks primarily support advertisement data spoofing and, to some extent, BDAs spoofing — both

essential for BLE impersonation. However, advertisement spoofing is non-trivial, as either the Android API or the Bluetooth chip constrains the extent to which advertisement data and occasionally the BDAs can be modified. In contrast, we observed that changing the BDAs alone suffices to trigger reconnection by a previously paired mobile phone — provided the spoofed device used a static BDAs. As users typically only see the device name, spoofing the name and BDAs can already constitute an effective attack. As we demonstrate in Fig. 5a, four of eight attacks were infeasible. Device information gathering was infeasible due to outdated dependencies (Zendehdel et al.) and no publicly available code (Siboni et al.). Fingerprinting failed as it was an announced but unimplemented feature in *BTLEmap*; Apple device information gathering and uptime measurement due to outdated prerequisites (*snout*). Across all considered tools — regardless of code availability — BDAs and

^hOnly if advertisement data does not exceed 31 bytes.

TABLE 7. Connection Establishment: BLE Prerequisites, Attacks and Results. ✓ Means That the Attack Was Fully or Almost Fully Possible, ✗ That the Attack Did Not Work, and n.a. That the Attack Could Not be Launched on This Device, Because it Supports Neither Legacy Pairing Nor Pairing in General.

Tool/Framework	Attacks		Test Devices			
	Sniffing of Pairing Handshake	TK Cracking	Smart Ring I	Smart Ring II	Muse S	OneTouch R113
ubertooth	✗		n.a.	n.a.	n.a.	✓
crackle		✗	n.a.	n.a.	n.a.	✗
WHAD	✗		n.a.	n.a.	n.a.	✓
		✗	n.a.	n.a.	n.a.	✗
nRF Sniffer for BLE	✗		n.a.	n.a.	n.a.	✓

advertisement spoofing emerged as the most widely supported attacks.

Table 7 presents the results of the attacks in the **connection establishment** phase which was only the cracking of the Legacy Pairing handshake after recording. Since recording and cracking require different tools or frameworks, we included both in Table 7. As Table 7 shows, we successfully recorded the pairing handshake using all tested tools and frameworks (ubertooth, WHAD, nRF Sniffer for Bluetooth LE); however, cracking the TK and decrypting the exchanged messages failed with both (crackle and WHAD). As noted in Section VI, only the OneTouch R113 performs pairing; therefore, we tested the relevant tools and frameworks exclusively on this device. Since we successfully installed and ran all tools and frameworks, we omit a distribution view in Fig. 5.

Table 8 summarizes the attack results of the third phase, the **connection between an attacker and a legitimate device**. All tested tools and frameworks successfully performed Service Discovery and could either read or fuzz the test devices. Five out of eight could also perform GATT Server spoofing. Packet tampering is only possible with the WHAD framework. Some attacks could not be executed: discovering a device's minimum security level (ATT-Profiler), due to a runtime error, and DoS attacks, due to missing publicly available code (Siboni et al., Bluetooth Low Energy Devices Security Testing Framework). The WHAD framework also supports packet tampering and replay, although these were mostly unsuccessful in our tests. Packet relay is supported by both WHAD and bluetoothctl, with the latter showing successful results. Fig. 5b illustrates the attack distribution across all tools and frameworks identified in this work. As shown, GATT Service Discovery and reading, writing, notifying, or fuzzing—followed by GATT Server Spoofing—are the most widely supported attacks. Among the overall set of tools and frameworks identified in this work, GATT Service Discovery and reading, writing, notifying, or fuzzing — followed by GATT Server Spoofing — were the most widely supported attacks, aligning with evaluated tools. Packet relay, tampering and replay is mostly supported by tools and frameworks that we could not run successfully.

Our final table, Table 9, displays the results for attacks on **connected legitimate devices**. Sniffing traffic between

two legitimate devices succeeded using the nRF Sniffer for Bluetooth LE or the Bluetooth HCI Snoop Log. In contrast, the Ubertooth One performed only partially. High packet transmission rates caused issues, and missing sender, receiver, and characteristic information in the pcap file further hindered usability. Packet injection using the WHAD framework failed, as did connection jamming and session hijacking (BtleJack), the latter due to outdated hardware. We could not evaluate the final attack – message frequency analysis – due to outdated prerequisites (snout). Fig. 5c supports the findings in Table 9 sniffing remains the most widely supported feature across all identified tools and frameworks.

C. SUMMARY AND OUTLOOK

Overall, most attacks supported by the still-functional tools and frameworks are the same as those supported by all tools and frameworks we identified, regardless of whether they are still functional. These include BDAs and advertisement spoofing (advertising), sniffing of the pairing handshake — classified as a prerequisite rather than an attack — (connection establishment), GATT Service Discovery and reading, writing, notifying, or fuzzing, followed by GATT Server Spoofing (connection attacker - legitimate device), and sniffing (connection between legitimate devices). In summary, spoofing, sniffing, and basic GATT operations appear easier to implement or more commonly needed, which may explain their inclusion in most tools and frameworks. Fig. 1 shows that a few additional attacks remain feasible, including device tracking, proximity estimation, and packet relay. In total, four of five prerequisites and eight of 21 attacks remain possible using the tools and frameworks we identified as still usable. The evaluation on four devices cannot capture the full heterogeneity of BLE ecosystems. Results should therefore be interpreted as a feasibility assessment rather than exhaustive generalization. As multiple tools or frameworks can often support the same attack, we do not recommend a single tool per attack. However, our results show that any Linux distribution is a strong starting point for testing BLE devices, as most tools and frameworks were developed for this OSes. Although the WHAD framework failed in roughly half of our tests, we still consider it a powerful tool — not only due to active maintenance but also because it

TABLE 8. Connection Between an Attacker and a Legitimate Device: BLE Prerequisites, Attacks and Results. ✓ Means That the Attack Was Fully or Almost Fully Possible, ~ That it Was Partially Possible and ✗, That the Attack Did Not Work.

Tool/ Framework	Attacks								Test Devices			
	GATT Serv. Disc.	read/ write/ fuzz	Min. Sec. Level	GATT Serv. Spoof.	Pkt. Relay	Pkt. Tamp.	Pkt. Replay	DoS	Smart Ring I	Smart Ring II	Muse S	OneTouch R113
bluetoothctl	✗								✓	✓	✓	✓
(read)		✗							✓	✓	✓	✓
(automatically)				✗					~	~	~	~
(manually)				✗					~	~	~	~
					✗				✓ ⁱ	✓ ⁱ	✓ ⁱ	✓ ⁱ
EXPLIoT	✗								✓	✓	✓	✓
(fuzz)		✗							n.a. ^j	n.a. ^j	✓	✓
HomePwn	✗								~	~	~	~
(read)		✗							~	~	~	~
WHAD	✗								✓	✗	✓	✓
(read)		✗							✓	✓	✓	✓
(man., wble-periph)				✗					✓	✓	✓	✓
(wble-proxy)				✗					✗	✗	✗	✗
(wble-spawn)				✗					✓	✗	✗	✗
					✗				✓	✗	✗	✗
						✗			✗	✗	✗	✗
							✗		✗	✗	✗	✗
bettercap	✗								✓	✓	✓	✓
(read)		✗							~ ^k	~ ^k	~ ^k	~ ^k
nRF Connect Desktop	✗								✓	✓	✓	✓
(read)		✗							✗	✗	✗	✗
				✗					✗	✗	✗	✗
nRF Connect Mobile	✗								✓	✓	✓	✓
(read)		✗							✓	✓	✓	✓
				✗					✗	✗	✗	✗
LightBlue	✗								~	~	~	~
(read)		✗							✓	✓	✓	✓
(automatically)				✗					~	~	~	~
(manually)				✗					~	~	~	~

TABLE 9. Connection Between Two Legitimate Devices: BLE Attacks and Results. ✓ Means That the Attack Was Fully or Almost Fully Possible, ~ That it Was Partially Possible and ✗, That the Attack Did Not Work.

Tool/Framework	Attacks						Test Devices			
	Sniffing	Pkt. Inject.	Conn. Jamming	Session Hijack.	Msg. Freq.		Smart Ring I	Smart Ring II	Muse S	OneTouch R113
ubertooth	✗						~	~	~	~
WHAD			✗				✗	✗	✗	✗
nRF Sniffer for BLE	✗						✓	✓	✓	✓
Bluetooth HCI Snoop Log	✗						✓	✓	✓	✓

allows users to implement custom BLE functions or attacks [110].

VIII. CONCLUSION

We began by developing a taxonomy that classifies threats to BLE, BLE Mesh, and BLE privacy and localization and

ⁱOnly tested relay of read commands; Characteristics Descriptors could not be read.

^jThe device did not expose any characteristic with a known meaning (e.g., Device Name). We therefore did not perform fuzzing, as we could not verify whether it had any effect.

^kNo value decoding possible when the value's meaning is decoded in hexadecimal

their prerequisites, if applicable, into four phases. We then introduced low-cost hardware sniffers, tools, frameworks, and Linux packages and utilities for conducting attacks on BLE (excluding BLE Mesh and BLE privacy and localization) that we partly used to construct our taxonomy. Next, we conducted an in-depth evaluation of the attacks each tool or framework could perform. Finally, we summarized our findings and presented relevant statistics. In our empirical evaluation, we found that most attacks supported by still-functional tools and frameworks are also supported by all tools and frameworks we identified. However, only four out of five prerequisites and eight out of 21 attacks remain

feasible with the tools and frameworks we found to be still usable. The evaluation on four devices cannot capture the full heterogeneity of BLE ecosystems. Results should therefore be interpreted as a feasibility assessment rather than exhaustive generalization. Another key finding is that actively maintained code — i.e., with a last commit in 2024 or 2025 — remains functional, unlike older code last updated before 2024. As multiple tools or frameworks can often support the same attack, we do not recommend a single tool per attack. However, our results show that any Linux distribution is a strong starting point for testing BLE devices, as most tools and frameworks were developed for this OSes. Although the WHAD framework failed in roughly half of our tests, we still consider it a powerful tool — not only due to active maintenance but also because it allows users to implement custom BLE functions or attacks [110].

In summary it can be said that current BLE pentesting requires a patchwork of multiple, often unmaintained frameworks to realize all known attacks — this demonstrates an urgent need for a single, extensible, actively maintained framework that allows attacks to be integrated modularly.

ACKNOWLEDGMENT

The authors thank Anna Jansen (University Hospital Bonn) for creating the graphical abstract and Melissa Steininger (University Hospital Bonn) for advising on color selections to improve accessibility for readers with color-vision deficiencies.

They used AI (ChatGPT 4o and 5) to assist with tool and framework installation, language editing, grammar refinement, and generation of the Python code used for Fig. 3, Fig. 4, and the subfigures in Fig. 5.

REFERENCES

- [1] Lionel Sujay Vailshery. *Number of Internet of Things (IoT) Connected Devices Worldwide From 2019 to 2034, by Vertical (in Millions)*. Accessed: Aug. 14, 2025. [Online]. Available: <https://www.statista.com/statistics/1194682/iot-connected-devices-vertically/>
- [2] R. A. G. Srinivasan and T. Wang. (2025). *Next-Gen Patient Monitoring With Vivalink's Intelligent Biometrics Platform and AWS*. [Online]. Available: <https://aws.amazon.com/blogs/apn/next-gen-patient-monitoring-with-vivalinks-intelligent-biometrics-platform-and-aws/>
- [3] Qurasoft. *SaniQ OS*. Accessed: Jul. 26, 2023. [Online]. Available: <https://qurasoft.de/en/saniq/>
- [4] D. Riedlinger et al., *Development of an Early Warning System to Identify Patients at Risk of Falling—Combining the Analysis of Medication Prescription Data and Movement Profiles*. Berlin, Germany: Universitätsmedizin Berlin, Humboldt Univ. of Berlin, Technische Universität Berlin, 2024, pp. 108–113. [Online]. Available: <https://depositon.tu-berlin.de/items/f95d78e9-24ad-480f-bd2d-6beeff15a6e8>
- [5] (1996). *Health Insurance Portability and Accountability Act of 1996*. Accessed: Aug. 20, 2025. [Online]. Available: <https://www.govinfo.gov/content/pkg/CRPT-104hrpt736/pdf/CRPT-104hrpt736.pdf>
- [6] (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Accessed: Aug. 20, 2025. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679>
- [7] S. Moganedi and D. Pottas, "Identification of information security controls for fitness wearable manufacturers," in *Information and Cyber Security*, H. Venter, M. Looek, M. Coetzee, M. Eloff, J. Eloff, and R. Botha, Eds., Cham, Switzerland: Springer, 2020, pp. 112–128.
- [8] S.-C. Cha, J.-F. Chen, C. Su, and K.-H. Yeh, "A blockchain connected gateway for BLE-based devices in the Internet of Things," *IEEE Access*, vol. 6, pp. 24639–24649, 2018.
- [9] Modat. (2025). *Exposed to the Bare Bone: When Private Medical Scans Surface on the Internet*. [Online]. Available: <https://www.modat.io/post/1-million-healthcare-devices-exposed>
- [10] L. Hiciano. (2025). *Hospital Investigating Leak of 16.6 Million Documents*. [Online]. Available: <https://www.taipetimes.com/News/taiwan/archives/2025/03/07/2003833024>
- [11] H. Greß, B. Krüger, and E. Tischhauser, "The newer, the more secure? standards-compliant Bluetooth low energy man-in-the-middle attacks on fitness trackers," *Sensors*, vol. 25, no. 6, p. 1815, Mar. 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/25/6/1815>
- [12] K. Lounis and M. Zulkernine, "Attacks and defenses in short-range wireless technologies for IoT," *IEEE Access*, vol. 8, pp. 88892–88932, 2020.
- [13] S. Khanam, I. B. Ahmedy, M. Y. I. Idris, M. H. Jaward, and A. Q. B. M. Sabri, "A survey of security challenges, attacks taxonomy and advanced countermeasures in the Internet of Things," *IEEE Access*, vol. 8, pp. 219709–219743, 2020.
- [14] R. R. Krishna, A. Priyadarshini, A. V. Jha, B. Appasani, A. Srinivasulu, and N. Bizon, "State-of-the-art review on IoT threats and attacks: Taxonomy, challenges and solutions," *Sustainability*, vol. 13, no. 16, p. 9463, Aug. 2021.
- [15] S. Deb et al., "Securing the Internet of Medical Things (IoMT): Real-world attack taxonomy and practical security measures," 2025, *arXiv:2507.19609*.
- [16] V. Kondeti and H. Bahşi, "Mapping cyber attacks on the Internet of Medical Things: A taxonomic review," in *Proc. 19th Annu. Syst. Syst. Eng. Conf. (SoSE)*, 2024, pp. 84–91.
- [17] A. I. Newaz, A. K. Sikder, M. A. Rahman, and A. S. Uluagac, "A survey on security and privacy issues in modern healthcare systems: Attacks and defenses," *ACM Trans. Comput. Healthcare*, vol. 2, no. 3, pp. 1–44, Jul. 2021.
- [18] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi, "SoK: The long journey of exploiting and defending the legacy of king harald Bluetooth," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 2847–228066.
- [19] A. Barua, M. A. Al Alamin, M. S. Hossain, and E. Hossain, "Security and privacy threats for Bluetooth low energy in IoT and wearable devices: A comprehensive survey," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 251–281, 2022.
- [20] A. Lacava, V. Zottola, A. Bonaldo, F. Cuomo, and S. Basagni, "Securing Bluetooth low energy networking: An overview of security procedures and threats," *Comput. Netw.*, vol. 211, May 2022, Art. no. 108953.
- [21] M. Cäsar, T. Pawelke, J. Steffan, and G. Terhorst, "A survey on Bluetooth low energy security and privacy," *Comput. Netw.*, vol. 205, Mar. 2022, Art. no. 108712.
- [22] S. Shrestha, E. Irby, R. Thapa, and S. Das, "Sok: A systematic literature review of Bluetooth security threats and mitigation measures," in *Emerging Information Security and Applications*. Cham, Switzerland: Springer, 2022, pp. 108–127.
- [23] K. S. Niksirat, L. Velykoivanenko, N. Zufferey, M. Cherubini, K. Huguenin, and M. Humbert, "Wearable activity trackers: A survey on utility, privacy, and security," *ACM Comput. Surveys*, vol. 56, no. 7, pp. 1–40, Apr. 2024.
- [24] M. R. Ghorri, T.-C. Wan, and G. C. Sodhy, "Bluetooth low energy mesh networks: Survey of communication and security protocols," *Sensors*, vol. 20, no. 12, p. 3590, Jun. 2020.
- [25] Core Specification Working Group. (2024). *Bluetooth Core Specification 6*. [Online]. Available: https://www.bluetooth.org/DocMan/handlers/DownloadDoc.aspx?doc_id=588232
- [26] G. A. Zendehdel, R. Kaur, I. Chopra, N. Stakhanova, and E. Scheme, "Automated security assessment framework for wearable BLE-enabled health monitoring devices," *ACM Trans. Internet Technol.*, vol. 22, no. 1, pp. 1–31, Sep. 2021.
- [27] S. Siboni et al., "Security testbed for Internet-of-Things devices," *IEEE Trans. Rel.*, vol. 68, no. 1, pp. 23–44, Mar. 2019.
- [28] *GitHub: Blue_Hydra*. Accessed: Aug. 27, 2025. [Online]. Available: https://github.com/ZeroChaos/blue_hydra
- [29] J. Mikulskis, J. K. Becker, S. Gvozdenovic, and D. Starobinski, "Snout: An extensible IoT pen-testing tool," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 2529–2531.
- [30] A. Heinrich, M. Stute, and M. Hollick, "BTLEmap: Nmap for Bluetooth low energy," in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, Jul. 2020, pp. 331–333.
- [31] J. K. Becker, D. Li, and D. Starobinski, "Tracking anonymized Bluetooth devices," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 3, pp. 50–65, Jul. 2019.

- [32] (2025). *Manpage BTMGMT*. [Online]. Available: <https://www.kali.org/tools/bluez/#btmgmt>
- [33] C. Skallak and S. Schmidt, “Indescribably blue: Bluetooth low energy threat landscape,” in *Proc. 9th Int. Conf. Internet Things, Big Data Secur.*, 2024, pp. 339–350.
- [34] A. Ray, V. Raj, M. Oriol, A. Monot, and S. Obermeier, “Bluetooth low energy devices security testing framework,” in *Proc. IEEE 11th Int. Conf. Softw. Test., Verification Validation (ICST)*, Apr. 2018, pp. 384–393.
- [35] P. Gonz  les, J. E. Garc  a, and L. Fern  ndez. (2019). *Home-PWN: Swiss Army Knife for Pentesting of IoT Devices*. [Online]. Available: https://github.com/Telefonica/HomePWN/blob/master/Papers/%5BPAPER%5Dhomepwn_ENG.pdf
- [36] D. Cauquil and R. Cayre. (2025). *Welcome to WHAD’s Documentation!*. Accessed: May 9, 2025. [Online]. Available: <https://whad.readthedocs.io/en/latest/index.html>
- [37] S. Jasek. (2016). *Gattacking Bluetooth Smart Devices*. [Online]. Available: <https://www.blackhat.com/docs/us-16/materials/us-16-Jasek-GATTacking-Bluetooth-Smart-Devices-Introducing-a-New-BLE-Proxy-Tool-wp.pdf>
- [38] (2025). *BluetoothCTL-Advertise*. [Online]. Available: <https://salsa.debian.org/bluetooth-team/bluez/-/blob/debian/sid/client/bluetoothctl-advertise.rst>
- [39] Nordic Semiconductor. *NRF Connect for Mobile*. Accessed: Mar. 27, 2024. [Online]. Available: <https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile>
- [40] (2024). *NRF Connect Bluetooth Low Energy*. [Online]. Available: <https://docs.nordicsemi.com/bundle/nrf-connect-ble/page/index.html>
- [41] S. Teplov. (2025). *Wireshark Dissectors for Apple’s BLE Continuity Protocol—Readme*. Accessed: Aug. 4, 2025. [Online]. Available: <https://github.com/furiousMAC/continuity/tree/master/dissector>
- [42] M. L. Hale, K. Lotfy, R. F. Gamble, C. Walter, and J. Lin, “Developing a platform to evaluate and assess the security of wearable devices,” *Digit. Commun. Netw.*, vol. 5, no. 3, pp. 147–159, Aug. 2019.
- [43] M. Ryan. (2018). *Ubertooth-BTLE*. Accessed: Jul. 25, 2024. [Online]. Available: <https://github.com/greatscottgadgets/ubertooth/blob/master/host/doc/ubertooth-btle.md>
- [44] M. Ryan, “Bluetooth: With low energy comes low security,” in *Proc. 7th USENIX Workshop Offensive Technol. (WOOT 13)*, 2013, p. 4.
- [45] (2025). *Bluetoothctl-Gatt*. [Online]. Available: <https://salsa.debian.org/bluetooth-team/bluez/-/blob/debian/sid/client/bluetoothctl-gatt.rst>
- [46] (2024). *Introduction*. [Online]. Available: <https://www.bettercap.org/project/introduction/>
- [47] NCC Group Plc. *BLESuite CLI*. Accessed: Jul. 17, 2025. [Online]. Available: <https://github.com/nccgroup/BLESuite-CLI>
- [48] *Bluetooth LE*. Accessed: Feb. 11, 2025. [Online]. Available: <https://exploit.readthedocs.io/en/latest/tests/bluetooth.html>
- [49] Punch Through. *Lightblue: Your Ultimate BLE Debugging & Development Toolkit*. Accessed: Aug. 27, 2024. [Online]. Available: <https://punchthrough.com/lightblue/>
- [50] (2024). *GitHub: HomePWN*. [Online]. Available: <https://github.com/Telefonica/HomePWN>
- [51] M. E. Garbelini, C. Wang, S. Chattopadhyay, S. Sun, and E. Kurniawan, “SweynTooth: Unleashing mayhem over Bluetooth low energy,” in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC 20)*, 2020, pp. 911–925.
- [52] Core Specification Working Group. (2025). *Assigned Numbers*. [Online]. Available: <https://www.bluetooth.com/specifications/assigned-numbers/>
- [53] A. Gupta, *The IoT Hacker’s Handbook: A Practical Guide To Hacking the Internet of Things*. Cham, Switzerland: Springer, 2019.
- [54] P. Sivakumaran and J. B. Alis, “A low energy profile: Analysing characteristic security on BLE peripherals,” in *Proc. 8th ACM Conf. Data Appl. Secur. Privacy*, Mar. 2018, pp. 152–154.
- [55] Punch Through. (2025). *Virtual Devices Screen (iOS Only)*. Accessed: Jun. 11, 2025. [Online]. Available: <https://punchthrough.com/how-to-use-lightblue/#virtual-device>
- [56] M. L. Hale, D. Ellis, R. Gamble, C. Waler, and J. Lin, “SecuWear: An open source, multi-component hardware/software platform for exploring wearable security,” in *Proc. IEEE Int. Conf. Mobile Services*, May 2015, pp. 97–104.
- [57] *GitHub: BLE-Replay*. Accessed: Aug. 4, 2025. [Online]. Available: <https://github.com/nccgroup/BLE-Replay>
- [58] R. Goyal, N. Dragoni, and A. Spognardi, “Mind the tracker you wear: A security analysis of wearable health trackers,” in *Proc. 31st Annu. ACM Symp. Appl. Comput.*, 2016, pp. 131–136.
- [59] Y. Zhang, J. Weng, R. Dey, and X. Fu, *Bluetooth Low Energy (BLE) Security and Privacy*. Cham, Switzerland: Springer, 2019, pp. 1–12.
- [60] Nordic Semiconductor. (2025). *BLE-Sniffer Command Overview*. Accessed: Jul. 25, 2024. [Online]. Available: <https://docs.nordicsemi.com/bundle/nrfutil/page/nrfutil-ble-sniffer/guides/overview.html>
- [61] (2018). *BTLEJack: A New Bluetooth Low Energy Swiss-Army Knife*. Accessed: Aug. 20, 2025. [Online]. Available: <https://github.com/virtualabs/btlejack>
- [62] Mesh Working Group. (2023). *Mesh Protocol 1.1*. [Online]. Available: https://www.bluetooth.com/specifications/specs/html/?src=MshPRT_v1.1/out/en/index-en.html
- [63] Martin Woolley. (2019). *The Bluetooth? Mesh Networking Primer*. [Online]. Available: <https://www.bluetooth.com/bluetooth-mesh-networking-primer/>
- [64] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi, “Formal model-driven discovery of Bluetooth protocol design vulnerabilities,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 2285–2303.
- [65] L. Montecchiari, A. Trotta, L. Bononi, and M. Di Felice, “Bluetooth mesh technology for the joint monitoring of indoor environments and mobile device localization: A performance study,” in *Proc. IEEE 19th Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2022, pp. 193–199.
- [66] I. Natgunanathan, N. Fernando, S. W. Loke, and C. Weerasuriya, “Bluetooth low energy mesh: Applications, considerations and current state-of-the-art,” *Sensors*, vol. 23, no. 4, p. 1826, Feb. 2023.
- [67] A. Adomnicai, J. J. A. Fournier, and L. Masson, “Hardware security threats against Bluetooth mesh networks,” in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, May 2018, pp. 1–9.
- [68] T. Claverie and J. L. Esteves, “BlueMirror: Reflections on Bluetooth pairing and provisioning protocols,” in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 339–351.
- [69] M. J  rgens et al., “Bluetooth mesh networks for indoor localization,” in *Proc. 20th IEEE Int. Conf. Mobile Data Manage. (MDM)*, Jun. 2019, pp. 397–402.
- [70] F.   lvarez, L. Almon, A.-S. Hahn, and M. Hollick, “Toxic friends in your network: Breaking the Bluetooth mesh friendship concept,” in *Proc. 5th ACM Workshop Secur. Standardisation Res. Workshop*, Nov. 2019, pp. 1–12.
- [71] E. Kalinin, D. Belyakov, D. Bragin, and A. Konev, “IoT security mechanisms in the example of BLE,” *Computers*, vol. 10, no. 12, p. 162, Nov. 2021.
- [72] P. Locatelli, M. Perri, D. M. Jimenez Gutierrez, A. Lacava, and F. Cuomo, “Device discovery and tracing in the Bluetooth low energy domain,” *Comput. Commun.*, vol. 202, pp. 42–56, Mar. 2023.
- [73] M. Cominelli, P. Patras, and F. Gringoli, “Dead on arrival: An empirical study of the Bluetooth 5.1 positioning system,” in *Proc. 13th Int. Workshop Wireless Netw. Testbeds, Experim. Eval. Characterization*. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 13–20.
- [74] R. Zhang et al., “SADIF: Spoofing attack on BLE direction finding based localization system,” in *Proc. 26th Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.* New York, NY, USA: Association for Computing Machinery, Oct. 2025, pp. 61–70.
- [75] Straihte and Elizabeth. (2022). *Ubertooth Retirement*. Accessed: Mar. 27, 2024. [Online]. Available: <https://greatscottgadgets.com/tags/ubertooth/>
- [76] M. Ryan. (2025). *UBERTOOTH-BTLE 1 ‘Jul. 2018’ ‘Project Ubertooth’ ‘User Commands’*. [Online]. Available: <https://github.com/greatscottgadgets/ubertooth/blob/master/host/doc/ubertooth-btle.md>
- [77] *Bluefruit LE Sniffer—Bluetooth Low Energy*. Accessed: Mar. 27, 2024. [Online]. Available: <https://www.adafruit.com/product/2269>
- [78] Nordic Semiconductor. *NRF51822*. Accessed: Mar. 27, 2024. [Online]. Available: <https://www.nordicsemi.com/Products/nRF51822>
- [79] Nordic Semiconductor. *Bluetooth Low Energy (Bluetooth LE)*. Accessed: 16, Apr. 2025. [Online]. Available: <https://www.nordicsemi.com/Products/Wireless/Bluetooth-Low-Energy>
- [80] J. Padgett et al., “NIST special publication 800–121 revision 2—Guide to Bluetooth security,” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. 2 (Revision 2), 2017. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-121r2-upd1.pdf>
- [81] Nordic Semiconductor. *NRF Sniffer for Bluetooth LE*. Accessed: Mar. 27, 2024. [Online]. Available: <https://www.nordicsemi.com/Products/Development-tools/nRF-Sniffer-for-Bluetooth-LE>

- [82] Nordic Semiconductor. (2024). *NRF Sniffer for Bluetooth LE—Technical Documentation*. Accessed: Mar. 27, 2024. [Online]. Available: https://docs.nordicsemi.com/bundle/nrfutil/page/nrfutil-ble-sniffer/nrfutil-ble-sniffer_0.12.0.html
- [83] *BlueZ—About*. Accessed: Aug. 6, 2025. [Online]. Available: <https://www.bluez.org/about/>
- [84] (2024). *Index: Bluez.Git*. [Online]. Available: <https://git.kernel.org/pub/scm/bluetooth/bluez.git/commit/?id=b1eb2c4cd057624312e0412f6c4be000f7fc3617>
- [85] *Blue-Hydra*. Accessed: Aug. 21, 2025. [Online]. Available: <https://www.kali.org/tools/blue-hydra/>
- [86] (2024). *Ubertooth*. [Online]. Available: <https://www.kali.org/tools/ubertooth/>
- [87] M. Ryan. (2016). *Crackle*. Accessed: Feb. 11, 2025. [Online]. Available: <https://github.com/mikeryan/crackle>
- [88] D. Cauquil. (2018). *You'd Better Secure Your BLE Devices or We'll Kick Your Butts!*. Accessed: Mar. 18, 2024. [Online]. Available: <https://virtualabs.fr/defcon26/DC26-DamienCauquil-YoudBetterSecureYourBLEDevices.pdf>
- [89] (2014). *PTES Main Page*. [Online]. Available: https://www.pentest-standard.org/index.php/Main_Page
- [90] Bettercap. *Bluetooth Low Energy*. Accessed: Jul. 24, 2025. [Online]. Available: <https://www.bettercap.org/modules/ble/>
- [91] R. Sharpe, E. Warnicke, and U. Lamping. *Wireshark User's Guide: Version 4.5.0*. Accessed: May 12, 2025. [Online]. Available: <https://www.wireshark.org/download/docs/Wireshark%20User%27s%20Guide.pdf>
- [92] (2024). *LightBlue*. [Online]. Available: <https://apps.apple.com/us/app/lightblue/id557428110?mt=8>
- [93] Punch Through. (2022). *LightBlue*. Accessed: Nov. 18, 2024. [Online]. Available: https://play.google.com/store/apps/details?id=com.punchthrough.lightblueexplorer&hl=en_US
- [94] Android. *Verifying and Debugging*. Accessed: Mar. 28, 2024. [Online]. Available: https://source.android.com/docs/core/connect/bluetooth/verifying_debugging
- [95] J. Müllers, H. Greß, L. Haaga, and B. Krüger, "Sensorik am Krankenbett—Synchrone datenakquise für studien in der epileptologie," *Clin. Epileptol.*, vol. 37, p. 61, Apr. 2024.
- [96] Core Specification Working Group. (2019). *Core Specification Supplement 9*. [Online]. Available: https://www.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=480305&_gl=1
- [97] D. Cauquil and R. Cayre. (2025). *GitHub: WHAD*. Accessed: Aug. 21, 2025. [Online]. Available: <https://github.com/whad-team/whad-client>
- [98] (2025). *WFilter: Generic Packet Filtering and Processing Tool*. Accessed: Aug. 18, 2025. [Online]. Available: <https://whad.readthedocs.io/en/stable/cli/generic/wfilter.html>
- [99] (2025). *GitHub: WHAD—Shell*. Accessed: Aug. 11, 2025. [Online]. Available: <https://github.com/whad-team/whad-client/blob/main/whad/ble/cli/peripheral/shell.py>
- [100] (2025). *NRF52840-DONGLE*. [Online]. Available: <https://github.com/virtualabs/btlejack/issues/32>
- [101] (2025). *Does the Btlejack-FW-V2.Hex Working on NRF52840?*. [Online]. Available: <https://github.com/virtualabs/btlejack/issues/85>
- [102] (2024). *GATTacker: Scan Shows Unknown State*. Accessed: Jul. 16, 2025. [Online]. Available: <https://github.com/securing/gattacker/issues/38>
- [103] Android Developers. (2008). *Announcing the Android 1.0 SDK, Release 1*. [Online]. Available: <https://android-developers.googleblog.com/2008/09/announcing-android-10-sdk-release-1.html>
- [104] *Ubertooth One*. Accessed: Aug. 27, 2025. [Online]. Available: <https://greatscottgadgets.com/ubertoothone/>
- [105] Nordic Semiconductor. *NRF52840 Dongle*. Accessed: Aug. 27, 2025. [Online]. Available: <https://www.nordicsemi.com/Products/Development-hardware/nRF52840-Dongle>
- [106] M. Ryan. (2022). *Ubertooth*. [Online]. Available: <https://github.com/greatscottgadgets/ubertooth>
- [107] Wireshark Foundation. (Feb. 2025). *GitHub: Wireshark*. Accessed: Aug. 27, 2025. [Online]. Available: <https://github.com/wireshark/wireshark>
- [108] Apple. *App Store: NRF Connect for Mobile*. Accessed: Aug. 28, 2025. [Online]. Available: <https://apps.apple.com/us/app/nrf-connect-for-mobile/id1054362403>
- [109] Android Developers. (2025). *Configure on-Device Developer Options*. [Online]. Available: <https://developer.android.com/studio/debug/dev-options>
- [110] Quarkslab's blog. (2024). *Bluetooth Low Energy GATT Fuzzing*. [Online]. Available: <https://blog.quarkslab.com/bluetooth-low-energy-gatt-fuzzing.html>



HANNAH GREß received the bachelor's (B.Eng.) degree in audiovisual media from the University of Applied Sciences Stuttgart, in 2018, and the master's (M.Sc.) degree in computer science from Philipps-University Marburg, in 2022. She is currently pursuing the Ph.D. degree in computer science with the University Hospital Bonn/University of Bonn.



JONAS SCHREITER is currently pursuing the bachelor's (B.Sc.) degree in cyber security and privacy with the Bonn-Rhein-Sieg University of Applied Sciences. Since 2024, he has been a Student Research and Teaching Assistant for Michael Rademacher at the Bonn-Rhein-Sieg University of Applied Sciences.



MICHAEL RADEMACHER received the master's degree in computer science from the Bonn-Rhein-Sieg University of Applied Sciences, in 2014, and the Ph.D. (Dr.-Ing.) degree from the Technical University of Kaiserslautern, in 2019. His Ph.D. research focused on developing usable long-distance Wi-Fi networks to enhance internet connectivity in rural areas. Since 2019, he has been leading a research group at Fraunhofer FKIE, Bonn, Germany, specializing in security and scalability in wireless networks. In 2023, he was appointed Professor of embedded systems and networks with the Bonn-Rhein-Sieg University of Applied Sciences.



BJÖRN KRÜGER received the master's degree (Dipl.-Inform.) in computer science in 2006 and the Ph.D. (Dr. rer. nat.) degree in computer science in 2012. He studied computer science, mathematics, and physics at Bonn University. From 2012 to 2015, he was a Postdoctoral Researcher with Bonn University. In 2015, he joined Gokhale Method Enterprise Inc., Stanford, CA, USA, as the Director of Research and Product Development, where he led the development of wearable devices to monitor users' posture. In 2022, he was with the Cologne University of Applied Sciences, as a Professor of distributed media applications and technologies. Since 2023, he has been a Professor of personalized digital health and telemedicine with the University of Bonn and the University Hospital Bonn. He has been a member of the Faculty of Medicine and, additionally since 2025, of the Faculty of Mathematics and Natural Sciences. Since 2025, he has also been an Associated Principal Investigator in the life sciences and health area of the Lamarr Institute for Machine Learning and Artificial Intelligence.